

TABLE OF CONTENTS

SL NO	DESCRIPTION	PAGE NO
01	Introduction	01-02
1.1	CNN Algorithm And Introduction to CNN	03-12
1.2	Plant Diseases	13-18
1.3	Mineral Fertilizers	19
1.4	Nitrogen Fertilizers	20
1.5	Phosphorus Fertilizers	21
1.6	Potassium Fertilizers	22
1.7	IDE	23
02	Literature Survey	24-27
03	Project Overview	28
3.1	Objective	28
3.2	Motivation	29
04	Software And Hardware Requirements	30
4.1	Software Requirements	30
4.2	Hardware Requirements	30
4.3	Existing System	31
4.4	Proposed System	32-33
05	Detailed Design	34
5.1	Overall Architecture	34
5.2	Plant Deployment	35
5.3	Data Flow Diagram(DFD)	36
5.4	Plant Entity Relationship diagram(ER Diagram)	37
5.5	Plant Use Case Diagram	38
5.6	Working	39-40
06	Language Description	41
6.1	Python Programming	41
6.2	Guidelines For Python Code	42-43
6.3	Characteristics Of Python	44-45
07	Implementation	46
7.1	Screenshots	46-49
7.2	Sample Code	50-55
7.3	Frontend	56-59
7.4	Backend	60-62
08	Conclusion	63
09	Future Enhancement	64
10	Bibliography	65

CHAPTER 1

INTRODUCTION

This application plays a vital role in recognizing the crop's leaves image and finding out the disease cause and fertilize suggestions after comparing with the trained model using Convolution Neural Network helps to differentiate each image by converting them by process of matrix convolution. CNN represents an interesting method for adaptive image processing and forms a link between general feed-forward neural networks and adaptive filters. Any user who grows different crops may not know all the diseases caused by fungus and bacteria. By uploading, their affected crops leave users to benefit by understanding the disease name and cause then our client can suggest fertilizer by analyzing the composition between modeled images and user images. Our application creates a good environment for fertilizer suggestions and two more important processes where the user can have weather and soil reports.

Based on the location given by the user, a weather report is generated and a soil report contains the suggested crops to grow by the user based on the soil samples given by the user. The pre-processing required in a ConvNet is much lower as compared to other classification algorithms. The problem of efficient disease protection is closely associated with the problems of sustainable agriculture. Inexperienced pesticide usage can cause the event of long-term resistance of the pathogens, severely reducing the power to fight back. Timely and accurate diagnosis of plant diseases is one among the pillars of precision agriculture. It is crucial to stop unnecessary waste of monetary and other resources, thus achieving healthier production during this changing environment, appropriate and timely disease identification including early prevention has never been more important. There are several ways to detect plant pathologies. Some diseases do not have any visible symptoms, or the effect becomes noticeable too late to act, and in those situations, a classy analysis is obligatory. However, most diseases generate some quite manifestation within the visible spectrum, therefore the eye examination of a trained professional is that the prime technique adopted in practice for disease detection.

To achieve accurate disease diagnostics a plant pathologist should possess good observation skills in order that one can identify characteristic symptoms. Variations in symptoms indicated by diseased plants may lead to an unsuitable diagnosis since unprofessional gardeners and hobbyists could have more difficulties determining it than knowledgeable plant pathologist. an automatic system designed to assist identify plant diseases by the plant's appearance and visual symptoms might be of great help to amateurs within the gardening process and trained professionals as a confirm.

To achieve accurate disease diagnostics a plant pathologist should possess good observation skills in order that one can identify characteristic symptoms. Variations in symptoms indicated by diseased plants may lead to an unsuitable diagnosis since unprofessional gardeners and hobbyists could have more difficulties determining it than knowledgeable plant pathologist. an automatic system designed to assist identify plant diseases by the plant's appearance and visual symptoms might be of great help to amateurs within the gardening process and trained professionals as a confirmation system in disease diagnostics. One of the main parts of Neural Networks is Convolutional neural networks (CNN). CNNs use image recognition and classification in order to detect objects, recognize faces, etc.

They are made up of neurons with learnable weights and biases. Each specific neuron receives numerous inputs and then takes a weighted sum over them, where it passes it through an activation function and responds back with an output. CNNs are primarily used to classify images, cluster them by similarities, and then perform object recognition. Many algorithms using CNNs can identify Plant disease, faces, street signs, animals, etc. Plant diseases can cause great damage to agricultural crops by significantly reducing their production because they restrict the growth of crops and lead to poor quality of products. Due to its lower yield and fiber, biofuel crops as agriculture struggles to keep up with the world's rapidly increasing population. An existing method for the detection and identification of plant leaf diseases is observed with the naked eye . Additionally, plants must be monitored in a consistent manner to avoid the spread of disease. Diseases, pests, and other undesirable substances present in crops can cause a sharp decline in agricultural production . The impact of these dangerous factors on crops has a direct impact on the decline of the quality and quantity of crops. To combat, control, and mitigate the effects of biological organisms and diseases. However, most diseases generate some kind of manifestation in the visible spectrum, so the naked eye examination of a trained professional is the prime technique adopted in practice for plant disease detection.

The problem of efficient disease protection is closely associated with the problems of sustainable agriculture Inexperienced pesticide usage can cause the event of long-term resistance of the pathogens, severely reducing the power to fight back. Timely and accurate diagnosis of plant diseases is one among the pillars of precision agriculture. It is crucial to stop unnecessary waste of monetary and other resources, thus achieving healthier production during this changing environment, appropriate and timely disease identification including early prevention has never been more important. There are several ways to detect plant pathologies. Some diseases do not have any visible symptoms, or the effect becomes noticeable too late to act, and in those situations, a classy analysis is obligatory. However, most diseases generate some quite manifestation

1.1 CONVOLUTIONAL NEURAL NETWORK ALGORITHM

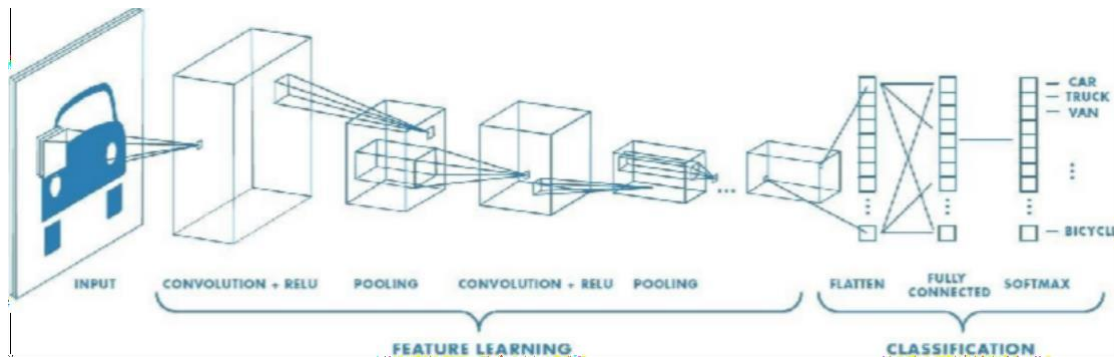


Fig 1.1: Feature Learning and Classification

Artificial Intelligence has been witnessing a monumental growth in bridging the gap between the capabilities of humans and machines. Researchers and enthusiasts alike, work on numerous aspects of the field to make amazing things happen. One of many such areas is the domain of Computer Vision.

The agenda for this field is to enable machines to view the world as humans do, perceive it in a similar manner and even use the knowledge for a multitude of tasks such as Image & Video recognition, Image Analysis & Classification, Media Recreation, Recommendation Systems, Natural Language Processing, etc. The advancements in Computer Vision with Deep Learning has been constructed and perfected with time, primarily over one particular algorithm-a Convolutional Neural Network.

They have applications in image and video recognition, recommender systems, image classification, image segmentation, medical image analysis, natural language processing, brain-computer interfaces, and financial time series. CNNs are regularized versions of multilayer perceptrons. Multilayer perceptron's usually mean fully connected networks, that is, each neuron in one layer is connected to all neurons in the next layer. The "full connectivity" of these networks makes them prone to over fitting data.

1.1 CNN INTRODUCTION

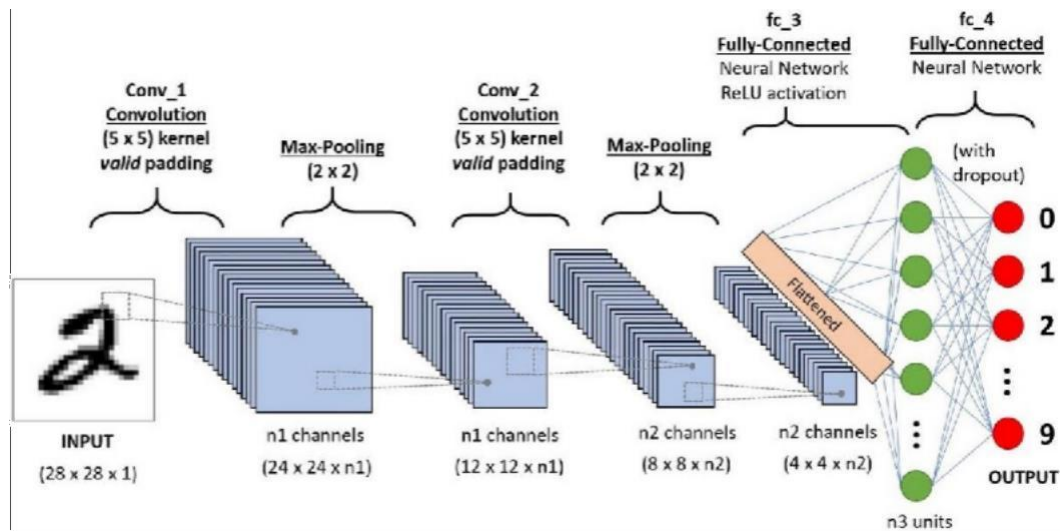


Fig 1.2: A CNN sequence to classify digits

A Convolutional Neural Network (ConvNet/CNN) is a Deep Learning algorithm which can take in an input image, assign importance (learnable weights and biases) to various aspects/objects in the image and be able to differentiate one from the other. The pre-processing required in a ConvNet is much lower as compared to other classification algorithms. While in primitive methods filters are hand-engineered, with enough training, ConvNets have the ability to learn these filters/characteristics.

The architecture of a ConvNet is analogous to that of the connectivity pattern of Neurons in the Human Brain and was inspired by the organization of the Visual Cortex. Individual neurons respond to stimuli only in a restricted region of the visual field known as the Receptive Field. A collection of such fields overlap to cover the entire visual area. It is a class of artificial neural network(ANN), most commonly applied to analyze visual imagery. They are also known as Shift Invariant or Space Invariant Artificial Neural Networks (SIANN), based on the shared-weight architecture of the convolution kernels or filters that slide along features and provide translate-equivariant responses known as feature maps. Counter-intuitively, most convolutional neural networks are only equivariant, as opposed to invariant, to translation.

They have applications in image and video recognition, recommender systems, image classification, image segmentation, medical systems, image classification, image segmentation, medical image analysis, natural language processing, brain-computer interface, and financial time series. CNN are regularized versions of multiplayer perceptron. Multiplayer perceptron usually mean fully connected networks that is, each neuron in one layer is connected to all neurons on the next layer. The “full connectivity” of these networks makes them prone to over fitting data .Typical ways of regularization, or preventing over fitting, include: penalizing parameters during training (such as weight decay) or trimming connectivity (skipped connections, dropout, etc.) CNNs take a different approach towards regularization: they take advantage of the hierarchical pattern in data and assemble patterns of increasing complexity using smaller and simpler patterns embossed in their filters. Therefore, on a scale of connectivity and complexity, CNNs are on the lower extreme. Convolutional networks were inspired by biological processes in that the connectivity pattern between neurons resembles the organization of the animal visual cortex. Individual cortical neurons respond to stimuli only in a restricted region of the visual field known as the receptive field. The receptive fields of different neurons partially overlap such that they cover the entire visual field. CNNs use relatively little pre-processing compared to other image classification algorithms.

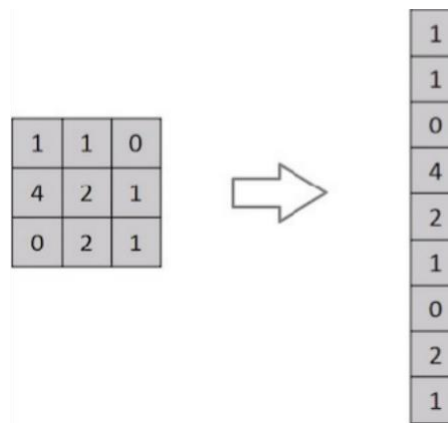


Fig 1.3: Flattering of a 3x3 image matrix into a 9x1 vector

An image is nothing but a matrix of pixel values. So flatten the image(e.g. 3x3 image matrix into a 9x1 vector) and feed it to a Multi-Level Perceptron for classification. In cases of extremely basic binary images, the method might show an average precision score while performing prediction of classes but would have little to no accuracy when it comes to complex images having pixel dependencies throughout. A ConvNet is able to **successfully capture the Spatial and Temporal dependencies** in an image through the application of relevant filters. The architecture performs a better fitting to the image dataset due to the reduction in the number of parameters involved and reusability of weights. In other words, the network can be trained to understand the sophistication of the image better.

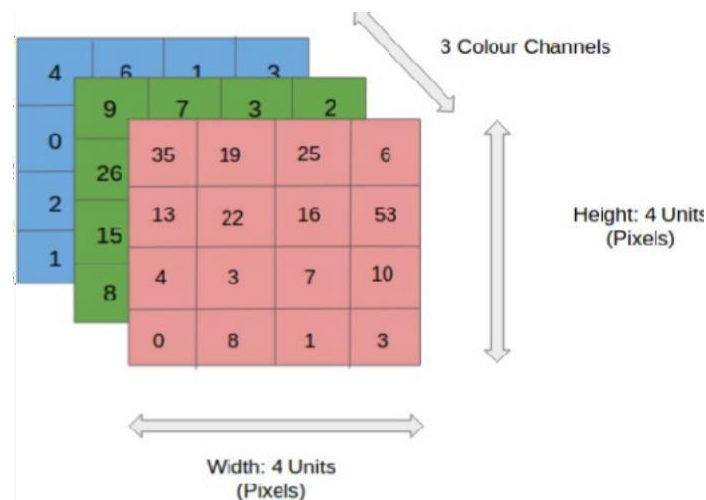


Fig 1.4: 4x4x3 RGB Image

In the above figure, we have an RGB image which has been separated by its three colour planes Red, Green, and Blue. There are a number of such colour spaces in which images exist — Grayscale, RGB, HSV, CMYK, etc. You can imagine how computationally intensive things would get once the images reach dimensions, say 8K (7680×4320). The role of the ConvNet is to reduce the images into a form which is easier to process, without losing features which are critical for getting a good prediction. This is important when we are to design an architecture which is not only good at learning features but also is scalable to massive datasets.

1 x1	1 x0	1 x1	0	0
0 x0	1 x1	1 x0	1	0
0 x1	0 x0	1 x1	1	1
0	0	1	1	0
0	1	1	0	0

Image

4		

Convolved
Feature

Fig 1.5: Convoluting a 5x5x1 image with a 3x3x1 kernel

Image Dimensions = 5 (Height) x 5 (Breadth) x 1 (Number of channels, eg. RGB). In the above demonstration, the green section resembles our **5x5x1 input image, I**. The element involved in carrying out the convolution operation in the first part of a Convolutional Layer is called the **Kernel/Filter, K**, represented in the color yellow. We have selected **K** as a **3x3x1 matrix**.

The Kernel shifts 9 times because of **Stride Length = 1 (Non-Strided)**, every time performing a **matrix multiplication operation between K and the portion P of the image** over which the kernel is hovering.

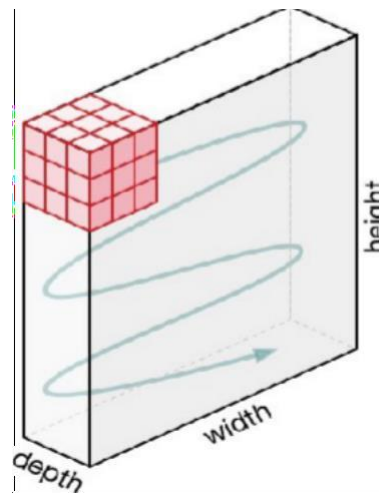


Fig 1.6: Movement of the kernel

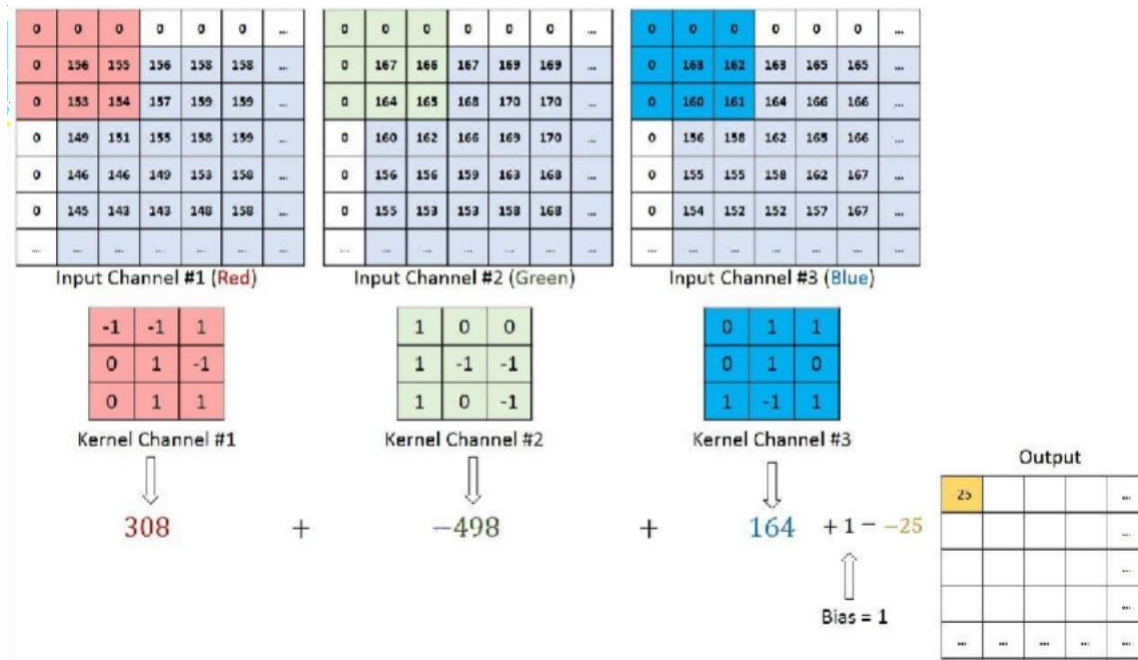


Fig 1.7: Convolution operation on a $M \times N \times 3$ image matrix with a $3 \times 3 \times 3$ Kernel

The filter moves to the right with a certain Stride Value till it parses the complete width. Moving on, it hops down to the beginning (left) of the image with the same Stride Value and repeats the process until the entire image is traversed.

In the case of images with multiple channels (e.g. RGB), the Kernel has the same depth as that of the input image. Matrix Multiplication is performed between K_n and I_n stack ($[K1, I1]; [K2, I2]; [K3, I3]$) and all the results are summed with the bias to give us a squashed one-depth channel Convolved Feature Output.

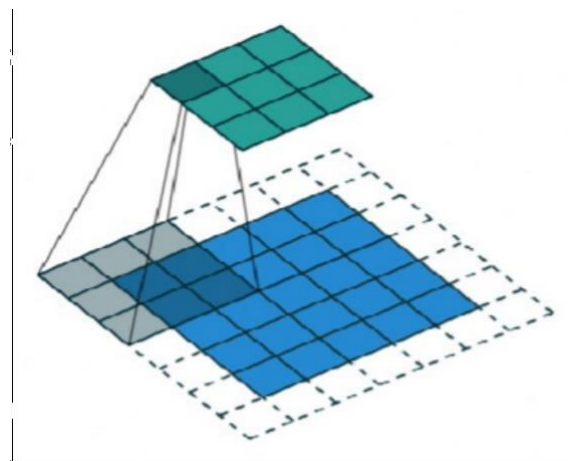


Fig 1.8: Convolution Operation with Stride Length = 2

The objective of the Convolution Operation is to **extract the high-level features** such as edges, from the input image. ConvNets need not be limited to only one Convolutional Layer. Conventionally, the first ConvLayer is responsible for capturing the Low-Level features such as edges, color, gradient orientation, etc.

With added layers, the architecture adapts to the High-Level features as well, giving us a network, which has the wholesome understanding of images in the dataset, similar to how we would. There are two types of results to the operation — one in which the convolved feature is reduced in dimensionality as compared to the input, and the other in which the dimensionality is either increased or remains the same. This is done by applying **Valid Padding** in case of the former, or **Same Padding** in the case of the latter.

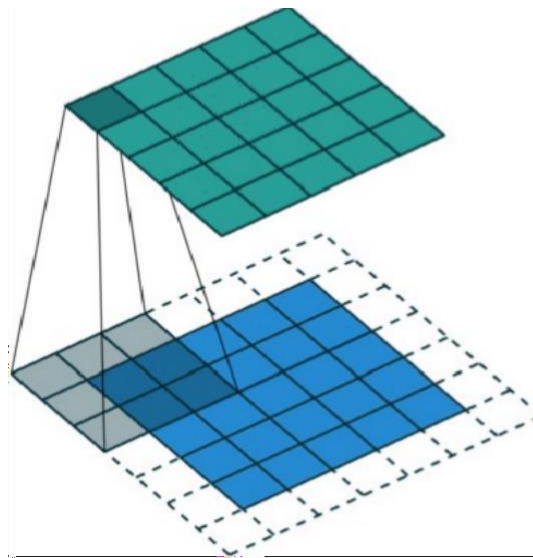


Fig 1.9: 5x5x1 image is padded with 0s to create a 6x6x1 image

When we augment the 5x5x1 image into a 6x6x1 image and then apply the 3x3x1 kernel over it, we find that the convolved matrix turns out to be of dimensions 5x5x1. Hence the name — **Same Padding**. On the other hand, if we perform the same operation without padding, we are presented with a matrix which has dimensions of the Kernel (3x3x1) itself — **Valid Padding**. The following repository houses many such GIFs which would help you get a better understanding of how Padding and Stride Length work together to achieve results relevant to our needs.

3.0	3.0	3.0
3.0	3.0	3.0
3.0	2.0	3.0

3	3	2	1	0
0	0	1	3	1
3	1	2	2	3
2	0	0	2	2
2	0	0	0	1

Fig 1.10: 3x3 polling over 5x5 convolved feature

Similar to the Convolutional Layer, the Pooling layer is responsible for reducing the spatial size of the Convolved Feature. This is to **decrease the computational power required to process the data** through dimensionality reduction. Furthermore, it is useful for **extracting dominant features** which are rotational and positional invariant, thus maintaining the process of effectively training of the model.

There are two types of Pooling: Max Pooling and Average Pooling. Max Pooling returns the maximum value from the portion of the image covered by the Kernel. On the other hand, Average Pooling returns the average of all the values from the portion of the image covered by the Kernel. Max Pooling also performs as a Noise Suppressant. It discards the noisy activations altogether and also performs de-noising along with dimensionality reduction. On the other hand, Average Pooling simply performs dimensionality reduction as a noise suppressing mechanism. Hence, we can say that Max Pooling performs a lot better than Average Pooling.

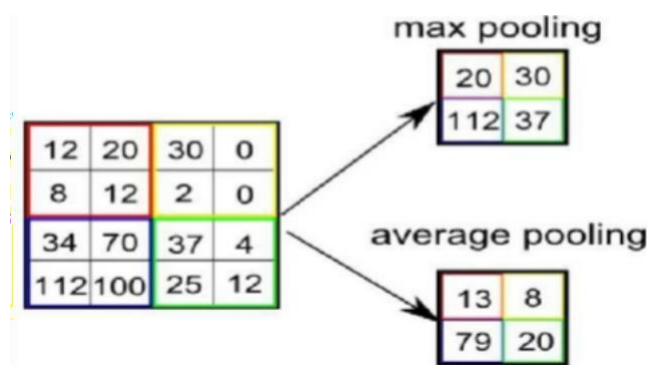


Fig 1.11 Types of Pooling

The Convolutional Layer and the Pooling Layer, together form the i-th layer of a Convolutional Neural Network. Depending on the complexities in the images, the number of such layers may be increased for capturing low-levels details even further, but at the cost of more computational power. After going through the above process, we have successfully enabled the model to understand the features. Moving on, we are going to flatten the final output and feed it to a regular Neural Network.

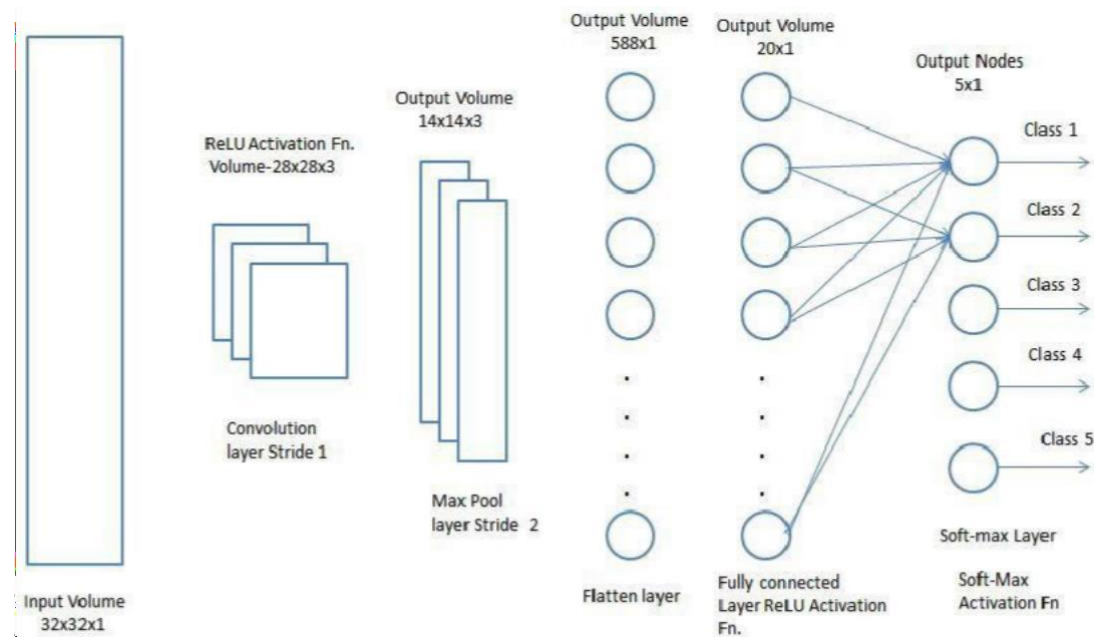


Fig 1.12: Classification-Fully Connected layer

Adding a Fully-Connected layer is a (usually) cheap way of learning non-linear combinations of the high-level features as represented by the output of the convolutional layer. The Fully-Connected layer is learning a possibly non-linear function in that space.

Now that we have converted our input image into a suitable form for our Multi-Level Perceptron, we shall flatten the image into a column vector. The flattened output is fed to a feed-forward neural network and backpropagation applied to every iteration of training. Over a series of epochs, the model is able to distinguish between dominating and certain low-level features in images and classify them using the **SoftMax Classification** technique

Texted Based Applications.

Further popular uses of Tensor Flow are, text-based applications such as sentimental analysis (CRM, social media), Threat Detection (Social Media, Government) and Fraud Detection (Insurance, Finance) Language Detection is one of the most popular uses of text based applications.

We all know Google Translate, which supports over 100 languages translating from one to another. The evolved versions can be used for many cases like translating jargon legalese in contracts into plain language. Text Summarization.

Image Recognition.

Mostly used by Social Media, Telecom and Handset Manufacturers; Face Recognition, Image Search, Motion Detection, Machine Vision and Photo Clustering can be used also in Automotive, Aviation and Healthcare Industries. Image Recognition aims to recognize and identify people and objects in images as well as understanding the content and context. Tensor Flow object recognition algorithms classify and identify arbitrary objects within larger images. This is usually used in engineering applications to identify shapes for modelling purposes (3D space construction from 2D images) and by social networks for photo tagging (Facebook's Deep Face).

By analyzing thousands of photos of trees for example, the technology can learn to identify a tree it has never seen before. Image recognition refers to the task of inputting an image into a neural network and having it output some kind of label for that image. The label that the network outputs will correspond to a pre-defined class. There can be multiple classes that the image can be labeled as, or just one. If there is a single class, the term "*recognition*" is often applied, whereas a multi-class recognition task is often called "*classification*". A subset of image classification is object detection, where specific instances of objects are identified as belonging to a certain class like animals, cars, or people. In order to carry out image recognition/classification, the neural network must carry out feature extraction. Features are the elements of the data that you care about which will be fed through the network. In the specific case of image recognition, the features are the groups of pixels, like edges and points, of an object that the network will analyze for pattern.

1.2 PLANT DISEASES

CEDER APPLE RUST (APPLE)



FIG 1.13 CEDER APPLE RUST

Cedar apple rust (*Gymnosporangium juniperi-virginianae*) is a fungal disease that requires juniper plants to complete its complicated two-year life-cycle. Spores overwinter as a reddish-brown gall on young twigs of various juniper species. In early spring, during wet weather, these galls swell and bright orange masses of spores are blown by the wind where they infect susceptible apple and crab-apple trees. The spores that develop on these trees will only infect junipers the following year.

From year to year, the disease must pass from junipers to apples to junipers again; it cannot spread between apple trees. On apple and crab-apple trees, look for pale yellow pinhead sized spots on the upper surface of the leaves shortly after bloom. These gradually enlarge to bright orange-yellow spots which make the disease easy to identify. Orange spots may develop on the fruit as well. Heavily infected leaves may drop prematurely. There are some ways by which we can provide treatment of Cedar apple rust:

Choose resistant cultivars when available.

Rake up and dispose of fallen leaves and other debris from under trees.

Remove galls from infected junipers. In some cases, juniper plants should be removed entirely.

Apply preventative, disease-fighting fungicides labeled for use on apples weekly, starting with bud break, to protect trees from spores being released by the juniper host. This occurs only once per year, so additional applications after this springtime spread are not necessary.

On juniper, rust can be controlled by spraying plants with a copper solution (0.5 to 2.0 oz/ gallon of water) at least four times between late August and late October.

Safely treat most fungal and bacterial diseases with SERENADE Garden. This broad-spectrum bio-fungicide uses a patented strain of *Bacillus subtilis* that is registered for organic use. Best of all, SERENADE is completely non- toxic to honey bees and beneficial insects.

Containing sulfur and pyrethrins, Bonide® Orchard Spray is a safe, one-hit concentrate for insect attacks and fungal problems. For best results, apply as a protective spray (2.5 oz/ gallon) early in the season.

Since the juniper galls are the source of the spores that infect the apple trees, cutting them is a sound strategy if there aren't too many of them. While the spores can travel for miles, most of the ones that could infect your tree **are within a few hundred feet**. The best way to do this is to prune the branches about 4-6 inches below the galls. You will want to disinfect your pruning shears, so you don't spread the infection. Dip them in 10% bleach or alcohol for at least 30 seconds between cuts.

If your tree has a history of infection with cedar apple rust, you will want to get ahead of the infection and take preemptive measures. This is critical in the spring, when the juniper galls are releasing their spores. The time to treat your tree is between the pink stage of the blossoms (when the leaves are turning green) to the period when the petals drop.

GRAY LEAF SPOT (CORN)



FIG 1.14 GRAY LEAF SPOT

Gray leaf spot is the number one disease in all corn production. The fungus survives the winter on residue from the prior corn crop, providing a primary source of the inoculum, particularly when growers plant continuous corn or use a no-till soybean/corn rotation. Spores from the fungus are splashed onto the lower leaves early in the season. When these resulting infections produce spores, the inoculum can be carried by air currents higher into the canopy or into neighboring fields. Mature lesions from gray leaf spot are rectangular, as they are restricted by the leaf veins.

As every veteran corn producer knows, corn is susceptible to a myriad of foliar fungal diseases. The most aggressive of these diseases, gray leaf spot, has been increasing in economic impact in many regions of the world over the past 10 years. Because gray leaf spot reduces a plant's photosynthetic surface area, the fungus significantly diminishes both the yield and quality of the crop. Field trials have shown potential yield losses from gray leaf spot range from 5 to 40 bu/A; however, losses as high as 100 percent are documented when early blighting of the leaves above the ear leaf occurs.

The planting of susceptible hybrids and weather conditions that favor rapid spread of the disease have contributed to the fact that gray leaf spot is now endemic in many regions of the world.

As reduced tillage became widespread in the 1980s and '90s, gray leaf spot disease grew commonplace in the heart of the Corn Belt. Farmers eager to capitalize on the increasing value and demand for corn augmented the amount of Midwest acreage in continuous corn. Both trends have contributed to an increase in the amount of corn residue in fields—a circumstance that can lead to severe infections the following year. The planting of susceptible hybrids and weather conditions that favor rapid spread of the disease have contributed to the fact that gray leaf spot is now endemic in many regions of the world.

Future yield losses will correlate with environmental conditions and the overwintering of the fungus in crop residue. If the humidity and frequency of heavy rainfall forecasted for many regions of the U.S. materializes, pathogens are expected to become even more problematic. Gray leaf spot initially appears as spots on the leaves that are round or oval, tan in color, and have a dark brown border. When the leaves are wet or humidity is high, the leaf spots turn gray and fuzzy with profuse spore production. In time, the leaf spots expand and girdle the leaf, causing it to die back from the tip. Significant damage to the turf stand may occur as the disease continues to progress.

EARLY BLIGHT (POTATO)



FIG 1.15 EARLY BLIGHT

Early blight of potato is caused by the fungal pathogen *Alternaria solani*. The disease affects leaves, stems and tubers and can reduce yield, tuber size, storability of tubers, quality of fresh-market and processing tubers and marketability of the crop.

In most production areas, early blight occurs annually to some degree. The severity of early blight is dependent upon the frequency of foliar wetness from rain, dew, or irrigation; the nutritional status of the foliage; and cultivar susceptibility. Across the Midwestern U.S., foliar infection is the most problematic phase of the disease, whereas in the western portion of the country, tuber infection can be more damaging. The disease first develops on mature and senescing foliage, and early maturing cultivars are the most susceptible. Potato is the primary host, but the disease also can be severe on tomatoes, and occur on other *solanaceous* plants such as hairy nightshade.

Symptoms of early blight occur on fruit, stem and foliage of tomatoes and stem, foliage and tubers of potatoes. Initial symptoms on leaves appear as small 1-2 mm black or brown lesions and under conducive environmental conditions the lesions will enlarge and are often surrounded by a yellow halo. Lesions greater than 10 mm in diameter often have dark pigmented concentric rings. This so-called “bullseye” type lesion is highly characteristic of early blight (Figure 4). As lesions expand and new lesions develop entire leaves may turn chlorotic and dehisce, leading to significant defoliation. Lesions occurring on stems are often sunken and lens-shaped with a light center, and have the typical concentric rings (Figure 5). On young tomato seedlings lesions may completely girdle the stem, a phase of the disease known as “collar rot,” which may lead to reduced plant vigor or death. Symptoms of early blight occur on fruit, stem and foliage of tomatoes and stem, foliage and tubers of potatoes. Initial symptoms on leaves appear as small 1-2 mm black or brown lesions and under conducive environmental conditions the lesions will enlarge and are often surrounded by a yellow halo (Figures 2 and 3). Lesions greater than 10 mm in diameter often have dark pigmented concentric rings. This so-called “bullseye” type lesion is highly characteristic of early blight.

TOMATO LATE BLIGHT



FIG 1.16 TOMATO LATE BLIGHT

Tomato late blight is caused by the oomycete pathogen *Pythophthora infestans* (*P. infestans*). The pathogen is best known for causing the devastating Irish potato famine of the 1840s, which killed over a million people, and caused another million to leave the country.

The first symptoms of late blight on tomato leaves are irregularly shaped, water-soaked lesions, often with a lighter halo or ring around them; these lesions are typically found on the younger, more succulent leaves in the top portion of the plant canopy. During high humidity, white cottony growth may be visible on underside of the leaf (Figure 2), where sporangia form .

Spots are visible on both sides of the leaves. As the disease progresses, lesions enlarge causing leaves to brown, shrivel and die. Late blight can also attack tomato fruit in all stages of development. Rotted fruit are typically firm with greasy spots that eventually become leathery and chocolate brown in color (Figure 4); these spots can enlarge to the point of encompassing the entire fruit. "Look alike" diseases: Other species of *Phytophthora*, specifically

P. nicotianae and *P. capsici*, can also cause lesions on tomato and tomato fruit similar to those caused by late blight. However, these two species are soilborne (as opposed to ariborne) and typically only cause disease after the plants have been flooded or if the plant had been in contact with wet soil. Late blight can be differentiated from early blight on tomato and other foliar fungal diseases by where they occur on the plant.

Leaf symptoms of late blight first appear as small, water-soaked areas that rapidly enlarge to form purple-brown, oily-appearing blotches. On the lower side of leaves, rings of grayish white mycelium and spore-forming structures may appear around the blotches. Entire leaves die and infections quickly spread to petioles and young stems. Infected fruit turn brown but remain firm unless infected by secondary decay organisms; symptoms usually begin on the shoulders of the fruit because spores land on fruit from above.

1.2 MINERAL FERTILIZERS

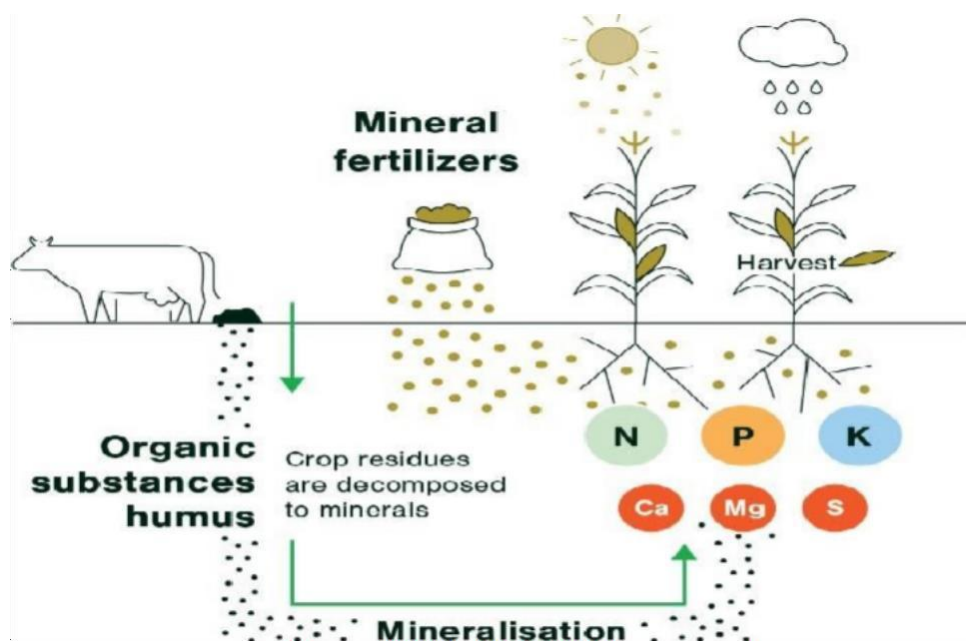


FIG 1.17 MINERAL FERTILIZERS

Mineral fertilizers are **materials, either natural or manufactured, containing nutrients essential for the normal growth and development of plants**. Plant nutrients are food for plants some of which are used directly for human food, others to feed animals, supply natural fibers or produce timber.

1.3 NITROGEN FERTILIZERS



FIG 1.18 NITROGEN FERTILIZERS

Nitrogen fertilizer means a substance containing nitrogen that is used for its plant nutrient content, is designed for use or claimed to have value in promoting plant growth, and requires a guaranteed analysis under Minnesota Statutes, section 18C.215. Nitrogen fertilizer does not include animal and vegetable manures that are not manipulated, or marl, lime, limestone, biosolids, industrial-by-product, industrial wastewater, irrigation water, or other products exempted by the commissioner.

It contains 24% Sulphur. Store well. It is applied at planting. Nitrogen will be released slowly. Takes about a month for all NH_4 to convert to NO_3 . The SO_4 radical combines with Ca and forms CaSO_4 radical combines with Ca and forms CaSO_4 which is soluble and hence Ca is removed & soil becomes acidic. It is Hygroscopic. So fertilizer bag should not be kept open for long time, it is most suitable for foliar spraying (1 to 5%). It undergoes hydrolysis, converted to Ammonium carbonate. Then to NH_4 and NO_3 . It is also used as food in small quantities for animals. In India it is manufactured to a larger extent. Highly hygroscopic, highly explosive thus all bags are tightly sealed until ready to use.

Keep all bags indoor in a dry place. Pile the stacks not higher than 6 bags. It is not manufactured in India. When it is allowed to be in stores it seeks into hard cake which requires to be broken up. It is more hygroscopic than Urea and ammonium nitrate. It is not suitable for paddy. It is not manufactured in India. Stable salt, stores well. Toxic to plants before it is converted into available form. Non leach able. It is a synthetic organic fertilizer.

1.4 PHOSPHORUS FERTILIZERS



FIG 1.19 PHOSPHORUS FERTILIZERS

Phosphorus is **an essential nutrient required for plant growth**. It helps in root development, plant maturation, and seed development. Along with nitrogen and potassium, phosphorus is one of the most important elements for plant life. The function of phosphorus in plants is very important.

It helps a plant convert other nutrients into usable building blocks with which to grow. Phosphorus is one of the main three nutrients most commonly found in fertilizers and is the “P” in the NPK balance that is listed on fertilizers. Phosphorus is essential to a plant’s growth, but what does it mean if you have high phosphorus in your soil, or a phosphorus deficiency? Keep reading to learn more about the importance of phosphorus in plant growth. The easiest way to tell is to look at the plants. If your plants are small, are producing little or no flowers, have weak root systems or a bright green or purplish cast, you have a phosphorus deficiency. Since most plants in the garden are grown for their flowers or fruit, replacing phosphorus in the soil if it is lacking is very important. Mineral phosphorus (P) fertilizers support high crop yields and contribute to feeding the teeming global population. However, complex edaphic processes cause P to be immobilized in soil, hampering its timely and sufficient availability for uptake by plants. The resultant low use efficiency of current water-soluble P fertilizers creates significant environmental and human health problems. Current practices to increase P use efficiency have been inadequate to curtail these problems.

1.5 POTASSIUM FERTILIZERS



FIG 1.20 POTASSIUM FERTILIZERS

Potassium is **the third key nutrient of commercial fertilizers**. It helps strengthen plants' abilities to resist disease and plays an important role in increasing crop yields and overall quality. Potassium fertilizers may also influence Cd availability to crops, through effects on plant growth, soil pH, ionic strength of the soil solution complexation, and/or ion competition (Grant et al., 1996). Under field conditions, KCl increased Cd concentrations in barley (Grant et al., 1996). The accompanying ion in the potassium fertilizer may have a strong effect on Cd availability. In pot studies with spring wheat, Cd accumulation and concentrations in plant shoots increased with increasing application rates of KCl and potassium sulfate (K_2SO_4) from 0 to 166 mg K kg^{-1} with no significant differences occurring between the two K sources (Zhao et al., 2004). However, shoot Cd concentrations were lower with KNO_3 than with KCl or K_2SO_4 and were only increased by the highest rate of KNO_3 application. In fields studies in Tasmania, Cd concentrations in potato tubers were higher with KCl application than with K_2SO_4 (Sparrow et al., 1994). As discussed in Section 3.1.2, Cl^- will increase the availability of Cd in soils through formation of chlorocomplexes (Smolders et al., 1998; Smolders and McLaughlin, 1996a,b). Therefore, application of Cl^- in KCl fertilizers may play an important role in its effect on Cd availability.

1.6 IDE (INTEGRATED DEVELOPMENT ENVIRONMENT)

An integrated development environment (IDE) or interactive development environment is a software application that provides comprehensive facilities to computer programmers for software development. An IDE normally consists of a source code editor, build automation tools and a debugger. Most modern IDEs offer intelligent code completion feature. Some IDEs contain a compiler, interpreter, or both, such as Net Beans and Eclipse, PyCharm; others do not, such as Sharp Develop and Lazarus.

The boundary between an integrated development environment and other parts of the broader *software development environment* is not well-defined. Sometimes a version control system and various tools are integrated to simplify the construction of a Graphical User Interface (GUI). Many modern IDEs also have a class browser, an object browser, and class diagram, for use in object-oriented software development. PyCharm is the most popular IDE used for Python scripting language. This chapter will give you an introduction to PyCharm and explains its features. PyCharm offers some of the best features to its users and developers in the following aspects

- Code completion and inspection, Advanced debugging, Support for web programming and frameworks such as Django and Flask. When coding in Python, queries are normal for a developers. You can check the last commit easily in PyCharm as it has the blue sections that can define the difference between the last commit and the current one. An IDE typically contains a code editor, a compiler or interpreter, and a debugger, accessed through a single graphical user interface (GUI). The user writes and edits source code in the code editor. The compiler translates the source code into a readable language that is executable for a computer. And the debugger tests the software to solve any issues or bugs. An IDE's toolbar looks much like a word processor's toolbar. The toolbar facilitates color-based organization, source- code formatting, error diagnostics and reporting, and intelligent code completion

CHAPTER 2

Literature Survey

In this section describes various approaches for detecting the disease in plant leaf using image processing technique **Ahmed, K., Shahidi, T.R & et al...[1]** As one of the top ten rice producing and consuming countries in the world, Bangladesh depends greatly on rice for its economy and for meeting its food demands.

To ensure healthy and proper growth of the rice plants it is essential to detect any disease in time and prior to applying required treatment to the affected plants. Since manual detection of diseases costs a large amount of time and labour, it is inevitable prudent to have an automated system. This paper presents a rice leaf disease detection system using machine learning approaches. Three of the most common rice plant disease namely leaf smut, bacterial leaf blight and brown spot disease are detected in this work. Clear images of affected rice leaves with white background were used as the input. After necessary pre-processing, the dataset was trained on with a range of different machine learning algorithms including that o KNN(K- Nearest Neighbors), J48(Decision Tree), Naïve Bayes and Logistic Regression. Decision tree algorithm, after 10-fold cross validation, achieved an accuracy of over 97% when applied on the dataset.

Ashraf, T., Khan, Y.N & et al.... [2] Weeds are normally sprayed uniformly across field irrespective of their density in the field. Automatic methods can be developed that help us to locate and spray weed according to its density. Automatic weed detection is necessary for precision spraying that can improve rice yield and reduce production cost. This aids farmers to get an idea of local weed infested areas of field. This paper proposes two classification techniques to distinguish images based on their weed density. The weed covered by our work is of category grass and is called nutgrass in rice. We aim to classify images into three classes according to grass density. The first technique uses texture features extracted from gray level cooccurrence matrix (GLCM) and produces an accuracy of 73% using Radial Basis Function (RBF) kernel in Support Vector Machine (SVM). Another technique proposed uses features such as moments that are invariant to scale and rotation to classify grass density. The Second technique outperforms the first one with an accuracy of 86% using Random Forest classifier.

Sanga, S.L., D. Machuve., Jomanga, K., & et al... [3] In Tanzania, smallholder farmers contribute significantly to banana production and Kagera, Mbeya, and Arusha are among the leading regions. However, pests and diseases are a threat to food security. Early detection of banana diseases is important to identify the diseases before too much damage is done on the plants. In this paper, a tool for early detection of banana diseases by using a deep learning approach is proposed. Five deep learning architectures, namely Vgg16, Resnet18, Resnet50, Resnet152 and InceptionV3 were used to develop models for banana diseases detection, achieving all high accuracies, varying from 95.41% for InceptionV3 to 99.2% for Resnet152. InceptionV3 was selected for mobile deployment because it demands much less memory. The developed tool was capable of detecting diseases with a confidence of 99% of the captured leaves from the real environment. This tool will help smallholder farmers conduct early detection of banana diseases and improve their productivity.

V.K Tewari, C.M Pareek, Gurdeep Lal, L.K Dhruw, Naseeb Singh & et al... [4] The agrochemical application with conventional sprayers results in wastage of applied chemicals, which not only increases the economic losses but also pollutes the environment. In order to overcome these drawbacks, an image processing based realtime variable-rate chemical spraying system was developed for the precise application of agrochemicals in diseased paddy crop based on crop diseases severity information. The developed system comprised of web cameras for image acquisition, laptop for image processing, microcontroller for controlling the system functioning, and solenoid valve assisted spraying nozzles. The chromatic aberration(CA) based image segmentation method was used to detect the diseased severity level of paddy plants, based on which the solenoid valves remained on for a specific time duration so that the required amount of agrochemical could be sprayed on the diseased paddy plants. Field performance of developed sprayer prototype was evaluated in the variable-rate application(VRA) and constant-rate application (CRA) modes. The field testing results showed a minimum 33.88% reduction in applied chemical while operating in the VRA mode as compared with the CRA mode. Hence, the developed system appears promising and could be used extensively to reduce the cost of pest management as well as to control environment pollution due to such agrochemicals.

Prabira Kumar Sethy, Nalini Kanta Barpanda, Amiya Kumar Rath, Santi Kumari Behera & et al... [5] Features are the vital factor for image classification in the field of machine learning. The advancement of deep convolutional neural network (CNN) shows the way for identification of rice diseases using deep features with the expectation of high returns. This technology introduced 5932 on-field images of four types of rice leaf diseases, namely bacterial blight, blast, brown spot and tungro. In addition, the performance evaluation of 11 CNN models in transfer learning approach and deep feature plus support vector machine (SVM) was carried out. The simulation results show the deep feature plus SVM perform better classification compared to transfer learning counterpart. Also, the performance of small CNN models such as mobilenetv2 and shuffle net was examined. The performance evaluation was carried out in terms of accuracy, sensitivity, specificity, false positive rate (FPR), F1 Score and training time. Again, the statistical analysis was performed to choose the better classification model.

Yang Li, Jiachen Yang & et al... [6] Xinjiang is the major cotton-producing area in China, also well known in the world for its high-quality cotton. The growth and quality of cotton are closely related to the pest attack, but it is difficult for farmers to manually recognize all the types of pests because of the similar appearance. To solve this problem, we propose a few-shot cotton pest recognition method, which only needs few raw training data, quite different from the typical deep learning methods. We use two datasets to verify the effectiveness and feasibility of the few-shot model, one is the National Bureau of Agricultural Insect Resources (NBAIR), the other is a dataset with the natural scenes. The convolutional neural network (CNN) is adopted to extract feature vectors of images. The CNN feature extractor is trained by the triplet loss to distinguish different pest species to ensure the system robustness. Furthermore, the few-shot recognition model is finally running in an embedded terminal, based on the compiled convolutional and max-pooling circuit in the FPGA and the control program in the ARM. The running speed reaches 2 frames per second and can be faster by the further parallelism in hardware.

Sue Han Lee, Herve Goeau , Pierre Bonnet, Alexis Joly & et al... [7] The control of plant diseases is a major challenge to ensure global food security and sustainable agriculture. Several recent studies have proposed to improve existing procedures for early detection of plant diseases through modern automatic image recognition systems based on deep learning . In this article , we study these methods in details, especially those based on convolutional neural networks. We first examine whether it is more relevant to fine-tune a pre-trained model on a plant identification task rather than a general object recognition task. In particular, we show, through visualization techniques, that the characteristics learned differ according to the approach adopted and that they do not necessarily focus on the part affected by the diseases. Therefore, we introduce a more intuitive method that considers diseases independently of crops, and we show that it is more effective than the classic crop-disease pair approach, especially when dealing with diseases involving crops that are not illustrated in the training database. This finding therefore encourages future research to rethink the current de facto paradigm of crop disease categorization.

Krishnaswamy Rangarajan Arvind, Purushothaman Raja& et al... [8] The biotic stress of agricultural crops is a major concern across the globe. Especially, its major effects are felt in economically poor countries where advanced facilities for diagnosis of a disease is limited as well as lack of awareness among the farmers. A recent revolution in smartphone technology and deep learning techniques have created an opportunity for automated classification of disease. In this study images acquired through smartphone are transmitted to a personal computer via wireless Local Area Network (LAN) for classification of ten different diseases using transfer learning in four major agricultural crops which are least explored. Six pre-trained a Convolutional Neural Network(CNN) have been used namely Alex Net, Visual Geometry Group 16(VGG16), VGG19, Google Net , ResNet101 and DenseNet201 with its corresponding results explored. Google Net resulted in the best validation accuracy of 97.3%. The misclassification was mainly due to Tobacco Mosaic Virus(TMV) and two-spotted spider mite. In test conditions, images were classified in real-time and prediction scores have been evaluated for each disease class. It depicted a reduction in accuracy in all models with VGG16 resulting in the best accuracy of 90%.

CHAPTER 3

PROJECT OVERVIEW

3.1 OBJECTIVES

These applications play a vital role in recognition the crop's leaves image and finding out the disease cause and fertilize suggestions after comparing with the trained model using Convolutional Neural Network helps to differentiate each image by converting them by process of matrix convolution. CNN represents an interesting method for adaptive image processing and forms a link between general feed-forward an interesting method for adaptive image processing and forms a link between general feed-forward networks and adaptive filters. Any user who grows different crops may not know all the diseases caused by fungus and bacteria. By uploading, their affected crops leave users to benefit by understanding the disease name and cause then our client can suggest fertilizer by analyzing the composition between modeled images and user images. application creates a good environment for fertilizer suggestions and two more important processes where the user can have weather and soil reports. Based on the location given by the user, a weather report is generated and a soil report contains the suggested crops to grow by the user based on the soil samples given by the user. The pre-processing required in a ConvNet is much lower as compared to other classification algorithms. Plant is exposed to many attacks from various micro- organism, bacterial disease and pests. The symptoms of the attacks are usually distinguished through the leaves, stem or fruit inspection. Disease that are commonly attack plants are Powdery Mildew and Leaf Blight and it may cause severe damaged if not controlled in early stages. The method is going to used in this project has widely being used for identification, detection, grading and quality inspection in the agriculture field. Detection and identification disease of a plant is very important especially, n producing an affected leaf.

3.2 MOTIVATION

Plant diseases are a very devastating threat to the agriculture industry and have the potential of pushing the whole of human society into starvation if not detected early. With the implementation of Convolutional Neural Network models, the detection of plant diseases will become easier and cheaper helping many farmers in the timely detection of plant diseases, preventing wastage of plants and protecting the transmission of diseases from diseased to healthy plants. A lot of the research that has been carried out on plant disease presents a comparative study using different CNN models but fail to explain the predictions made by their models.

In this project we not only provide a comparative study between the workings of a simple and complex model, but we also aim at providing explainability for the predictions made by the models. The motivation for including explainability lies in the fact that most of the CNN models widely used in this project for detecting the diseases of the leaf, which leads to the users using this model for predicting in not trusting and understanding how this CNN model works and make their predictions. The Artificial Intelligence techniques allow the users to understand their model predictions better and make decisions on their own as to whether, trust their model or not. The use of Django this server helps a great deal in the applications of plants disease detection as the transparency and explainability of the models being is vital in gaining the trust of the workers working in the agriculture industry as their livelihood is dependent on their production of healthy plants.

CHAPTER 4

HARDWARE AND SOFTWARE REQUIREMENTS:

4.1 Software Requirements

Programming Language:

Python 3.x (preferred for AI/ML projects).

Operating System:

Windows 10/11, Linux (Ubuntu), or macOS.

TensorFlow/Keras:

For building, training, and deploying the CNN model.

PyTorch:

Alternative framework for model development.

Libraries for Data Preparation & Processing:

OpenCV:

Used for reading, resizing, and preprocessing leaf images.

NumPy & Pandas:

For numerical operations and managing dataset data.

Scikit-learn:

For data preprocessing tasks like splitting into training/testing sets.

IDE/Editor:

Jupyter Notebook (for experimentation) or PyCharm.

Dataset Source:

PlantVillage dataset, Kaggle datasets, or custom collected data.

4.2 Hardware Requirements

Processor (CPU)

Multi-core processor such as Intel Core i5 / AMD Ryzen 5
 Required for running code, preprocessing images, and basic model training
 Even a normal laptop CPU can handle small CNN models

RAM (Memory)

Minimum: **8 GB RAM**

Recommended: **16 GB or more** for large datasets and smooth performance

Storage

At least **256 GB SSD** (recommended) or 1 TB HDD
Needed to store datasets, trained models, and software tools

Camera / Image Input Device

Smartphone camera or digital camera
Used to capture leaf images for disease detection

4.3 EXISTING SYSTEM

In general, critical factors for image analysis and pattern recognition, there are many models to initialize the process. To learn a robust distance metric, we need abundant side information which is usually unavailable in practice due to the high labelling cost. A process like a matrix factorization, distance metric learning, and Clustering method finds similar pixels to classify into clusters or classes, yet these distance metric learning problems in unsupervised or weakly supervised clustering tasks, especially for the unified theoretic schemes and optimization algorithms. There is no proper image recognition using a huge number of images trained into a model which can be achieved by a Convolution neural network. Basically, in existing system there is no such feature by which we can detect the affected leaf, for which there is huge cause of damage of different types of crops and plants. In recent times due to bad weather and less information about soil farmers facing huge loss. In crops different types of substances are present like diseases, pests, etc., The impact on these dangerous factors on crops has a appearance, morphology, and other characteristics of the leaves. By using the existing system it is impossible to find a affected leaf and cure for it .

4.4 PROPOSED SYSTEM

In the proposed system, any farmers who want to find the cause and disease formed in their crops are benefited. When the former upload leaves images of the affected crop by fungus and bacteria, the user cannot conclude what caused it and what fertilizer should be used to mitigate the problem in near future. Admin has a trained model which contains a huge amount of images by matrix conversion using CNN then admin compares the user given images with the trained model. So, initial images are converted into an input layer which is a grayscale image.

FEATURES

- Disease Detection.
- Image Recognition using CNN.
- ReLu (Rectifier Linear Unit).
- Fertilizer Suggestion.

TYPES OF DISEASE LEAF USED



Apple Leaf



Corn Leaf



Potato Leaf



Tomato Leaf

Fig. 3.1 4 Types of leaf diseases

CHAPTER 5

Detailed Design

5.1 OVERALL ARCHITECTURES

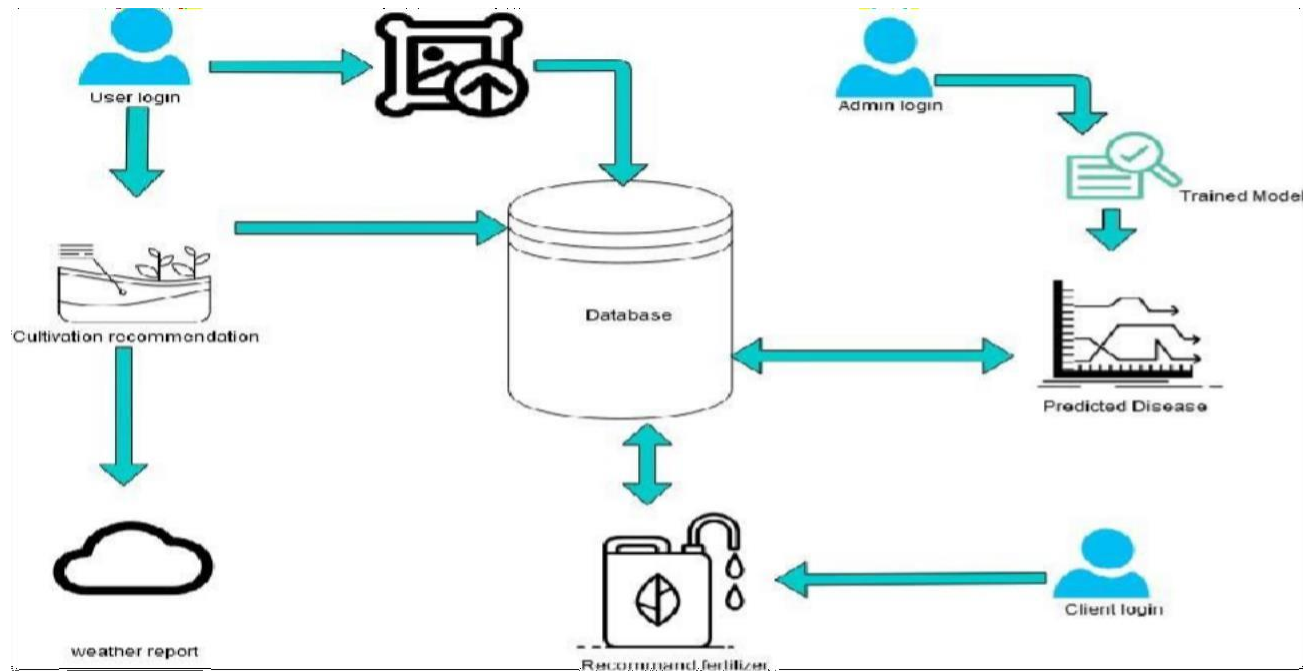


Fig. 4.0 ARCHITECTURE DIAGRAM

This diagram is the architecture of this project. Mainly user should login to upload the affected image of the leaf which is already uploaded in the MySQL Database. Then admin should login to predict disease of the image which will work through trained model and this trained model built by convolutional neural network algorithm which is basically a type of artificial neural network used in image recognition and processing that is specifically designed to process pixel data. After detecting the diseases of the leaf user should check with recommendation of cultivation and weather report which is most important for finding the place where crops can grow. Then for better fertilizer client should login and search for required fertilizer for crops and plants leaf.

5.2 PLANT DEPLOYMENT

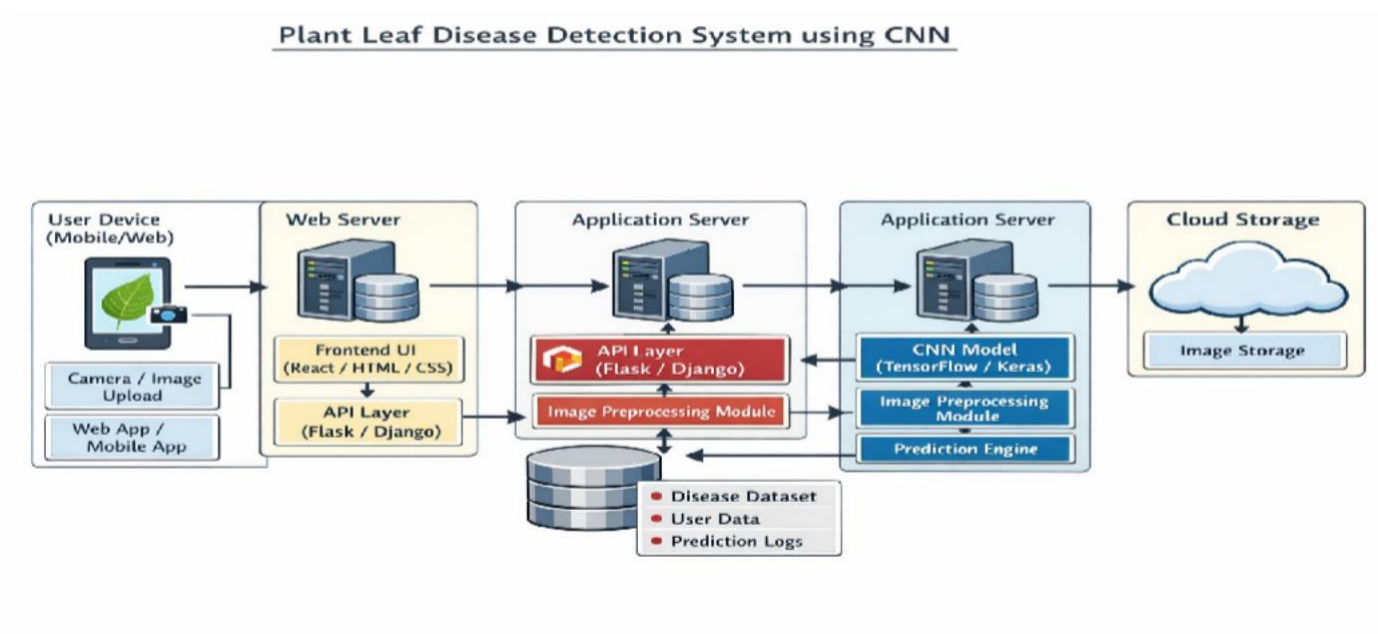


Fig. 4.1 PLANT DEPLOYMENT

The deployment diagram represents a **Plant Leaf Disease Detection system using a CNN model**, where the user captures or uploads a leaf image through a mobile or web application. This image is sent to the web server, which manages the user interface and forwards the request to the application server via an API.

The application server preprocesses the image and uses the CNN model to identify the disease. The prediction result is then sent back to the user, while important data such as images, user details, and prediction logs are stored in the database and cloud storage for future reference and analysis.

5.3 DATA FLOW DIAGRAM (DFD)

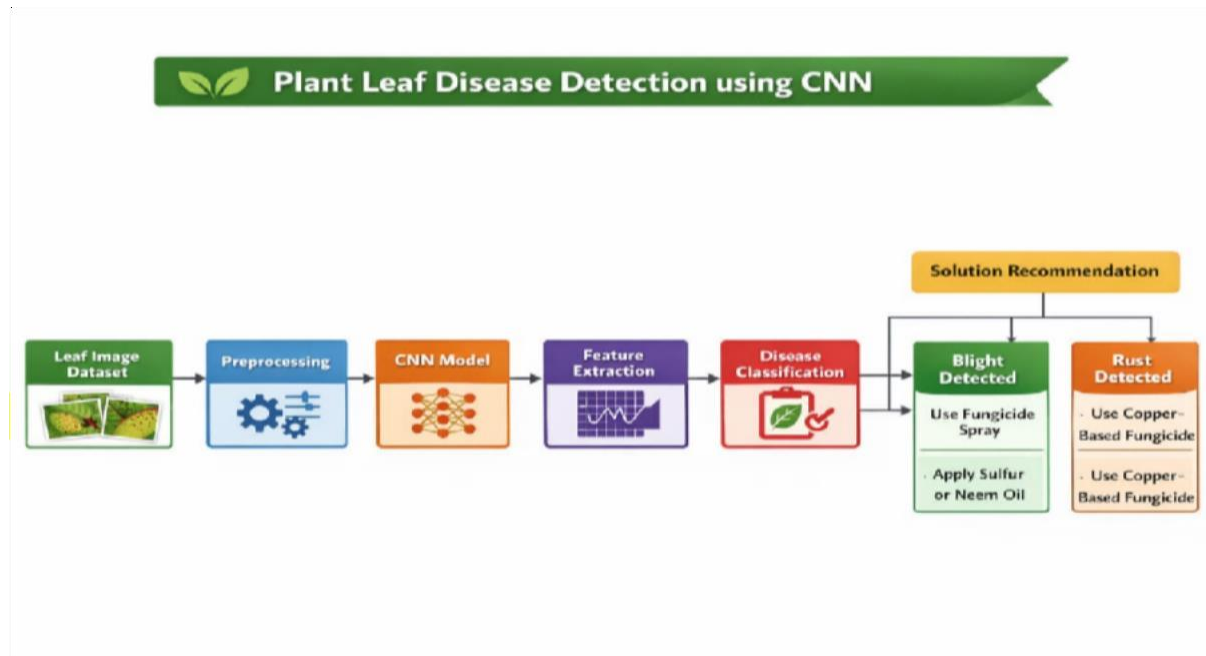


Fig. 4.2 DATA FLOW DIAGRAM

This diagram represents the workflow of a **Plant Leaf Disease Detection system using a CNN model**. It begins with a **leaf image dataset**, which contains images of healthy and diseased plant leaves. These images act as the input for the system and are essential for training and testing the model.

In the next step, **preprocessing** is performed on the images. This includes resizing, noise removal, normalization, and enhancement to improve image quality. Proper preprocessing ensures that the data is clean and suitable for accurate analysis by the model.

The processed images are then passed into the **CNN (Convolutional Neural Network) model**. The CNN automatically learns patterns such as color, texture, and shape from the images. This makes it highly effective for identifying disease-related features in plant leaves.

After that, **feature extraction and disease classification** take place. The model extracts important features from the images and classifies them into different disease categories, such as blight or rust, based on learned patterns.

Finally, the system provides **solution recommendations** based on the detected disease. For example, if blight is detected, it suggests using fungicide spray or neem oil, while for rust, it recommends copper-based fungicides. This helps farmers or users take appropriate action to protect their crops.

5.4 PLANT ENTITY RELATIONSHIP DIAGRAM

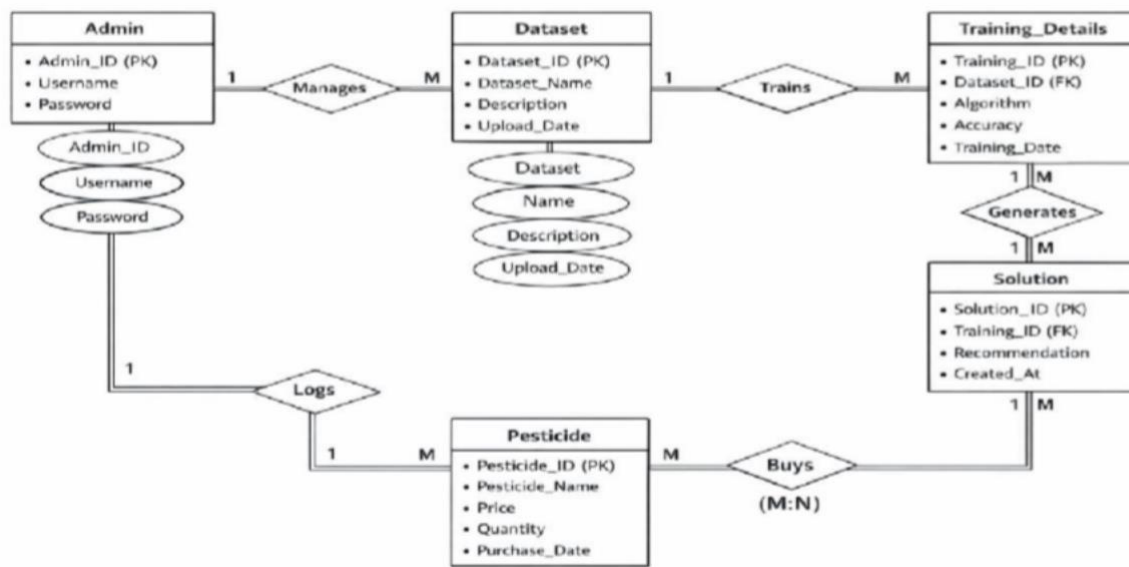


Fig. 4.3 ENTITY RELATIONSHIP DIAGRAM

This **Entity–Relationship (ER) diagram** represents the database design for a **Plant Leaf Disease Detection System**. It shows how different entities like admin, dataset, training details, solutions, and pesticides are related to each other.

The **Admin** entity contains attributes such as *Admin_ID* (*Primary Key*), *Username*, and *Password*. The admin is responsible for managing the system. There is a **one-to-many (1:M) relationship** between Admin and Dataset, labeled “*Manages*”, meaning one admin can manage multiple datasets, but each dataset is managed by only one admin.

The **Dataset** entity includes *Dataset_ID* (*PK*), *Dataset_Name*, *Description*, and *Upload_Date*. These datasets are used for training the model. The relationship “*Trains*” connects Dataset to **Training_Details** in a **one-to-many (1:M)** manner, indicating that one dataset can be used in multiple training processes.

The **Training_Details** entity stores information like *Training_ID* (*PK*), *Dataset_ID* (*FK*), *Algorithm*, *Accuracy*, and *Training_Date*. This entity records how the model is trained. It has a relationship “*Generates*” with the **Solution** entity (1:M), meaning one training process can generate multiple solutions or recommendations.

The **Solution** entity contains *Solution_ID* (PK), *Training_ID* (FK), *Recommendation*, and *Created_At*. These solutions represent the suggested treatments for detected diseases. The Solution entity is connected to the **Pesticide** entity through a **many-to-many (M:N) relationship** called “Buys”, indicating that multiple solutions can involve multiple pesticides and vice versa.

The **Pesticide** entity includes *Pesticide_ID* (PK), *Pesticide_Name*, *Price*, *Quantity*, and *Purchase_Date*. It stores details about pesticides used or recommended. Additionally, there is a “Logs” relationship involving Admin and Pesticide, which indicates that admins keep records or logs of pesticide-related activities.

Overall, this ER diagram clearly defines the structure of the system database, showing how data flows from dataset management to model training, solution generation, and pesticide usage, ensuring efficient organization and retrieval of information.

5.5 PLANT USE CASE DIAGRAM

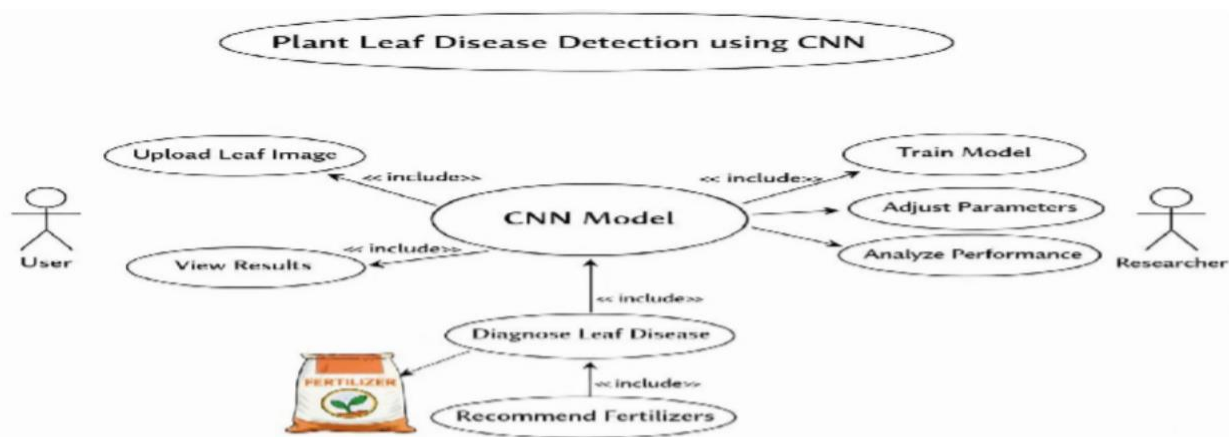


Fig. 4.4 USE CASE DIAGRAM

A use case diagram is a way to summarize details of a system and the users within that system. It is generally shown as a graphic depiction of interactions among different elements in a system. Use case diagrams will specify the events in a system and how those events flow, however, use case diagram does not describe how those events are implemented. A use case is a methodology used in system analysis to identify, clarify, and organize system requirements. In this context, the term "system" refers to something being developed or operated, such as a mail-order product sales and service Web site.

Use case diagrams are employed in UML (Unified Modeling Language), a standard notation for the modeling of real-world objects and systems. There are a number of benefits with having

diagrams such as flowcharts. System objectives can include planning overall requirements, validating a hardware design, testing and debugging a software product under development, creating an online help reference or performing a consumer-service-oriented task. For example, use cases in a product sales environment would include item ordering, catalog updating, payment processing, and customer relations. A use case diagram contains four components. The boundary, which defines the system of interest in relation to the world around it. The actors, usually individuals involved with the system defined according to their roles. The use cases, which are the specific roles played by the actors within and around the system. The relationships between and among the actors and the use cases. The updated use case diagram for **Plant Leaf Disease Detection using CNN** represents an intelligent system that not only detects plant diseases but also provides fertilizer recommendations. At the center of the system is the CNN model, which acts as the core processing unit. The user interacts with the system by uploading a leaf image, which is then analyzed by the CNN model to identify whether the plant is healthy or affected by a disease. Once the analysis is complete, the system displays the results to the user. This entire process is essential and interconnected, as indicated by the <<include>> relationships in the diagram.

A key enhancement in this version is the addition of the “Recommend Fertilizers” functionality. After diagnosing the leaf disease, the system suggests suitable fertilizers or treatments based on the identified condition. This makes the system more practical and useful for farmers, as it not only detects the problem but also provides a solution. Meanwhile, the researcher plays an important role in improving the system by training the model with datasets, adjusting parameters, and analyzing performance to increase accuracy. Overall, the system functions as a smart agricultural tool that combines disease detection with actionable recommendations, making it highly beneficial for real-world applications.

5.6 WORKING

Artificial Intelligence has been witnessing a monumental growth in bridging the gap between the capabilities of humans and machines. A Convolutional Neural Network (ConvNet/CNN) is a Deep Learning algorithm that can take in an input image, assign importance (learnable weights and biases) to various aspects/objects in the image, and be able to differentiate one from the other. The pre-processing required in a ConvNet is much lower as compared to other classification algorithms. In this project image segregation and identification is done using matrix factorization and machine learning. But we create a trained model using a Convolutional Neural Network where the user-given image is detected by converting and comparing the images which are in matrix form. In our application, the user image is compared with our trained model to identify the specifics of images and sent to the client where machine learning helps to find the right fertilizers for the crop

application use weather report for any given location and soil report - suggestion for the cultivation of specific crop for best productivity. Our application plays a vital role in recognizing the crop's leaves image and finding out the disease cause and fertilize suggestions after comparing with the trained model using Convolution Neural Network helps to differentiate each image by converting them by process of matrix convolution. CNN represents an interesting method for adaptive image processing and forms a link between general feed-forward neural networks and adaptive filters. Any user who grows different crops may not know all the diseases caused by fungus and bacteria. By uploading, their affected crops leave users to benefit by understanding the disease name and cause then our client can suggest fertilizer by analyzing the composition between modeled images and user images. Our application creates a good environment for fertilizer suggestions and two more important processes where the user can have weather and soil reports. Based on the location given by the user, a weather report is generated and a soil report contains the suggested crops to grow by the user based on the soil samples given by the user.

CHAPTER 6

LANGUAGE DESCRIPTION

6.1 PYTHON PROGRAMMING

Python is an interpreted language and not a compiled one, although compilation is a step. Python code, written in **.py** file is first compiled to what is called byte code which is stored with a **.pyc** or **.pyo** format. Instead of translating source code to machine code like C++, Python code is translated to byte code. This byte code is a low-level set of instructions that can be executed by an interpreter. One popular advantage of interpreted languages is that they are platform-independent.

As long as the Python byte code and the Virtual Machine have the same version, Python byte code can be executed on any platform (Windows, macOS, etc). Dynamic typing is another advantage. In static-typed languages like C++, you have to declare the variable type and any discrepancy like adding a string and an integer is checked during compile time. In older programming languages, memory allocation was quite manual. Many times when you use variables that are no longer in use or referenced anywhere else in the program, they need to be cleaned from the memory. Garbage Collector does that for you.

HOW PYTHON INTERPRETER WORKS

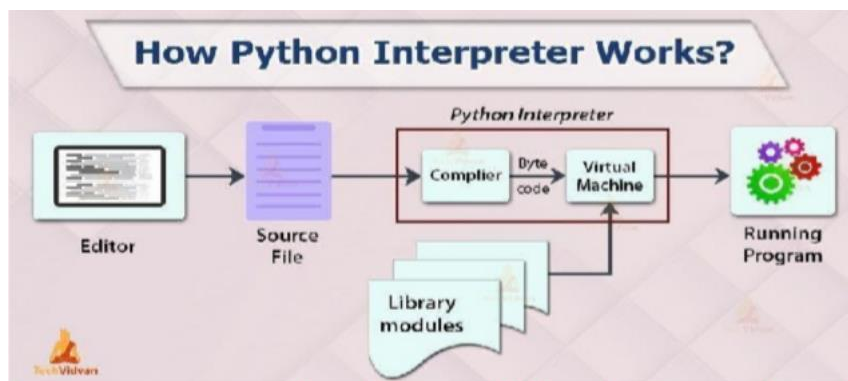


Fig 4.5 HOW PYTHON INTERPRETER WORKS

Python interpreter is **responsible for executing the Python scripts**. Python interpreter is a piece of software which works between the Python program and computer hardware. Here we are describing the series of ways to run Python scripts. The operating system command-line or terminal.

We write the python code in any text editor and save the same file using the “.py” extension in our system. Now, how will this code run? There must be some application or program like “python” or “python3” that must be installed in your system, and this is their duty to run this python code. This type of program is called Interpreter. Python does provide a python shell, which is used to execute a single command in python and display the result accordingly. The interpreter’s line-editing features include interactive editing, history substitution and code completion on systems that support the GNU Readline library. Perhaps the quickest check to see whether command line editing is supported is typing Control-P to the first Python prompt you get. If it beeps, you have command line editing; see Appendix Interactive Input Editing and History Substitution for an introduction to the keys. If nothing appears to happen, or if ^P is echoed, command line editing isn’t available; you’ll only be able to use backspace to remove characters from the current line.

6.2 GUIDELINES FOR PYTHON CODE

One of Guido’s key insights is that code is read much more often than it is written. The guidelines provided here are intended to improve the readability of code and make it consistent across the wide spectrum of Python code. As Python Enhancement Proposal says, “Readability counts”. A style guide is about consistency. Consistency with this style guide is important. Consistency within a project is more important. Consistency within one module or function is the most important. However, know when to be inconsistent – sometimes style guide recommendations just aren’t applicable. When in doubt, use your best judgment. Look at other examples and decide what looks best. And don’t hesitate to ask! In particular: do not break backwards compatibility just to comply with this PEP! For example, do not rely on CPython’s efficient implementation of in-place string concatenation for statements in the form `a += b` or `a = a + b`. This optimization is fragile even in CPython (it only works for some types) and isn’t present at all in implementations that don’t use refcounting.

In performance sensitive parts of the library, the `‘.join ()` form should be used instead. This will ensure that concatenation occurs in linear time across various implementations. Comparisons to singletons like `None` should always be done with `is` or `is not`, never the equality operators. Also, beware of writing `if x` when you really mean `if x is not None` – e.g., when testing whether a variable or argument that defaults to `None` was set to some other value. The other value might have a type (such as a container) that could be false in a Boolean context!

The first form means that the name of the resulting function object is specifically ‘f’ instead of the generic ‘<lambda>’. This is more useful for tracebacks and string representations in general. The use of the assignment statement eliminates the sole benefit a lambda expression can offer over an

explicit `def` statement (i.e. that it can be embedded inside a larger expression) Derive exceptions from `Exception` rather than `BaseException`.

Direct inheritance from `BaseException` is reserved for exceptions where catching them is almost always the wrong thing to do. Design exception hierarchies based on the distinctions that code *catching* the exceptions is likely to need, rather than the locations where the exceptions are raised. Aim to answer the question “What went wrong?” programmatically, rather than only stating that “A problem occurred” Class naming conventions apply here, although you should add the suffix “Error” to your exception classes if the exception is an error. Non-error exceptions that are used for non-local flow control or other forms of signalling need no special suffix.

Use exception chaining appropriately. `raise X from Y` should be used to indicate explicit replacement without losing the original traceback. When deliberately replacing an inner exception (using `raise X from None`), ensure that relevant details are transferred to the new exception (such as preserving the attribute name when converting `KeyError` to `AttributeError`, or embedding the text of the original exception in the new exception message). When catching exceptions, mention specific exceptions whenever possible instead of using a bare `except` clause. A bare `except:` clause will catch `SystemExit` and `KeyboardInterrupt` exceptions, making it harder to interrupt a program with `Control-C`, and can disguise other problems. If you want to catch all exceptions that signal program errors, use `except Exception:` (bare `except` is equivalent to `except BaseException:`). A bare `except:` clause will catch `SystemExit` and `KeyboardInterrupt` exceptions, making it harder to interrupt a program with `Control-C`, and can disguise other problems. If you want to catch all exceptions that signal program errors, use `except Exception:` (bare `except` is equivalent to `except BaseException:`). If the exception handler will be printing out or logging the traceback; at least the user will be aware that an error has occurred. If the code needs to do some cleanup work, but then lets the exception propagate upwards with `raise`, `try...finally` can be a better way to handle this case.

When catching operating system errors, prefer the explicit exception hierarchy introduced in Python 3.3 over introspection of `errno` values. Be consistent in return statements. Either all return statements in a function should return an expression, or none of them should. If any return statement returns an expression, any return statements where no value is returned should explicitly state this as `return None`, and an explicit return statement should be present at the end of the function.

6.3 CHARACTERISTICS OF PYTHON

We briefly list some of C's characteristics that define the language and also have led to its popularity as a programming language. Naturally we will be studying many of these aspects throughout the course.

Python is a dynamic, high level, free open source and interpreted programming language. It supports object- oriented programming as well as procedural oriented programming.

In Python, we don't need to declare the type of variable because it is a dynamically typed language.

Characteristics of Python:

There are many features in Python, some of which are discussed below –

Python is a high-level programming language. Python is very easy to learn the language as compared to other languages like C, C#, JavaScript, Java, etc...

One of the key features of python is Object-Oriented programming. Python supports object-oriented language and concepts of classes, objects encapsulation, etc.

Python is a high-level language. When we write programs in python, we do not need to remember the system architecture, nor do we need to manage the memory.

Python is an **Extensible** language. We can write us some Python code into C or C++ language and also, we can compile that code in C/C++ language.

Python language is also a portable language. For example, if we have python code for windows and if we want to run this code on other platforms such as Linux, Unix, and Mac then we do not need to change it, we can run this code on any platform.

Python is also an integrated language because we can easily integrated python with other languages like c, c++, etc.

Python is an Interpreted Language because Python code is executed line by line at a time. like other languages C, C++, Java, etc. there is no need to compile python code this makes it easier to debug our code. The source code of python is converted into an immediate form called **bytecode**.

Python has a large standard library which provides a rich set of module and functions so you do not have to write your own code for every single thing. There are many libraries present in python for such as regular expressions, unit-testing, web browsers, etc.

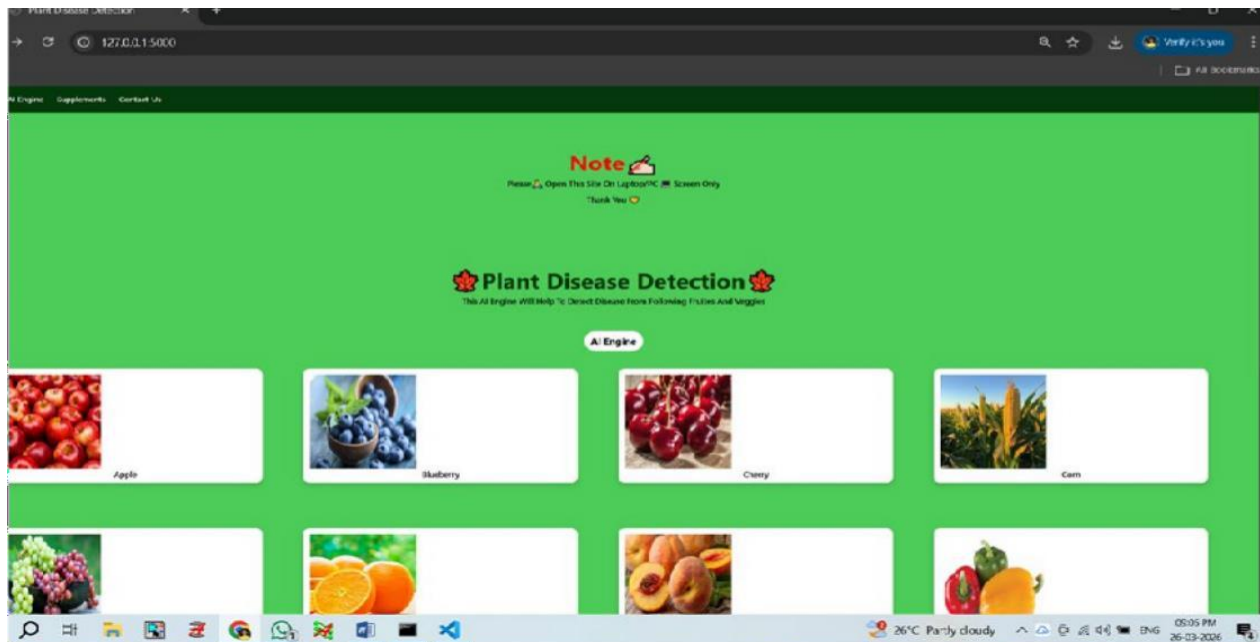
Python is a dynamically-typed language. That means the type (for example- int, double, long, etc.) for a variable is decided at run time not in advance because of this feature we don't need to specify the type of variable.

CHAPTER 7

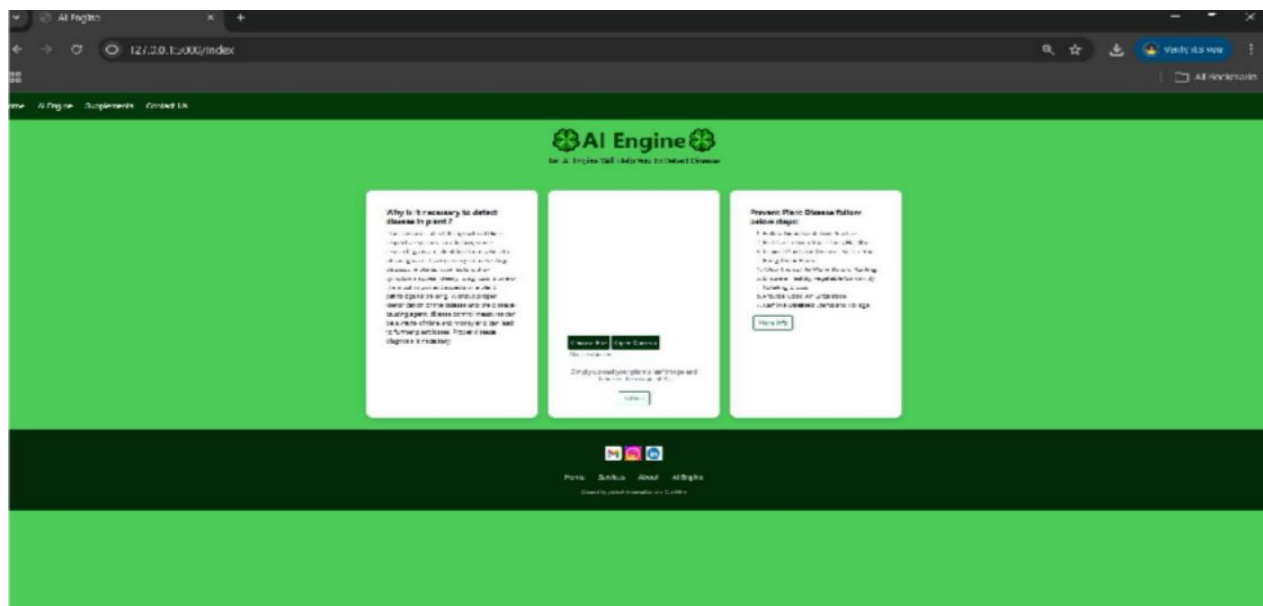
IMPLEMENTATION

7.1 SCREENSHOTS

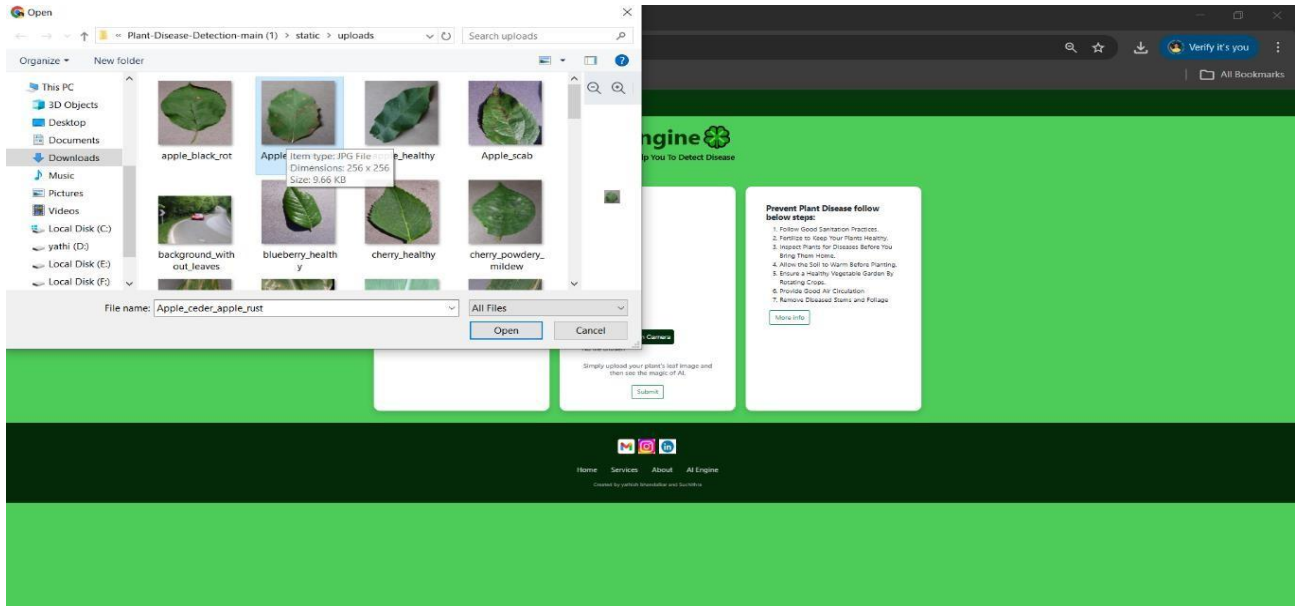
LOGIN PAGE



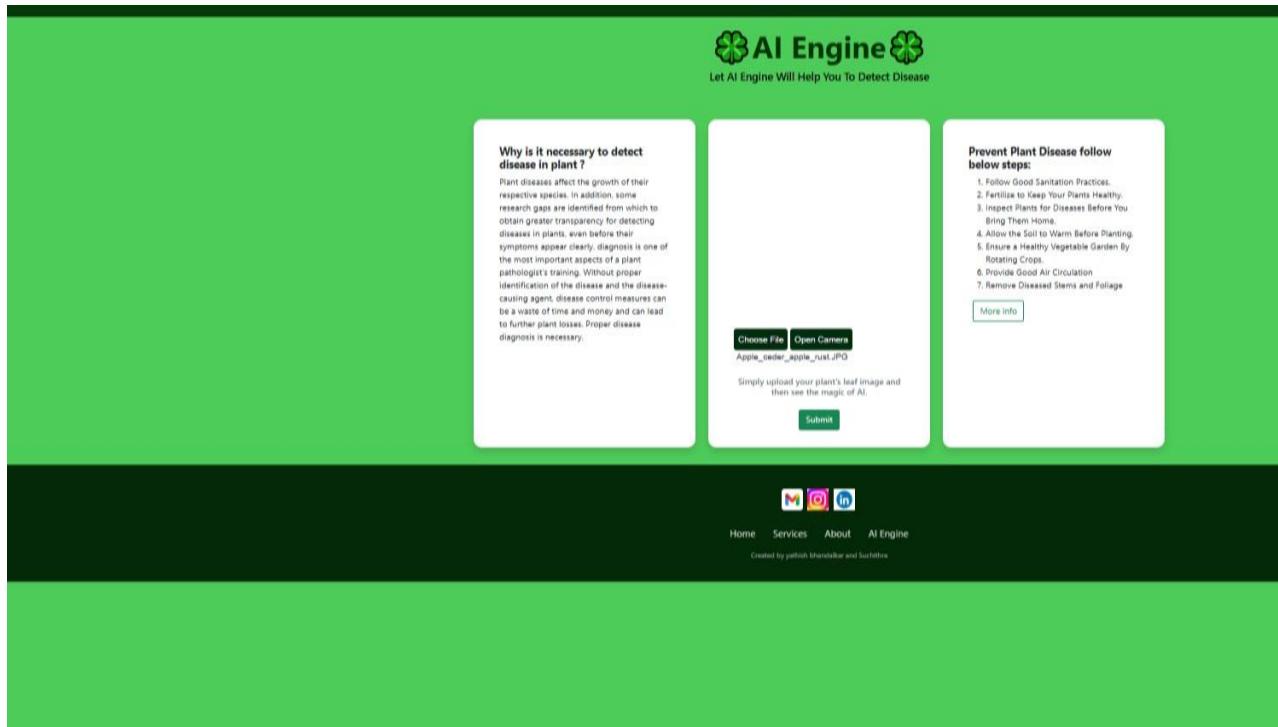
AI ENGINE



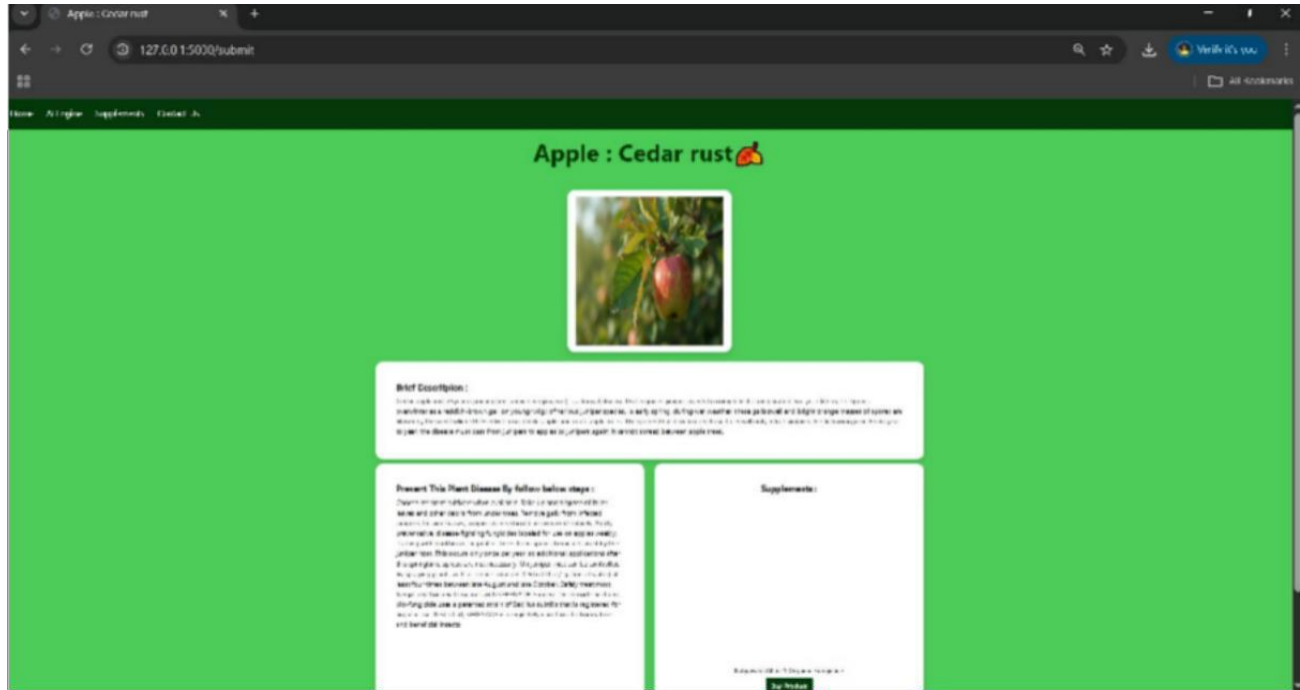
UPLOAD PAGE



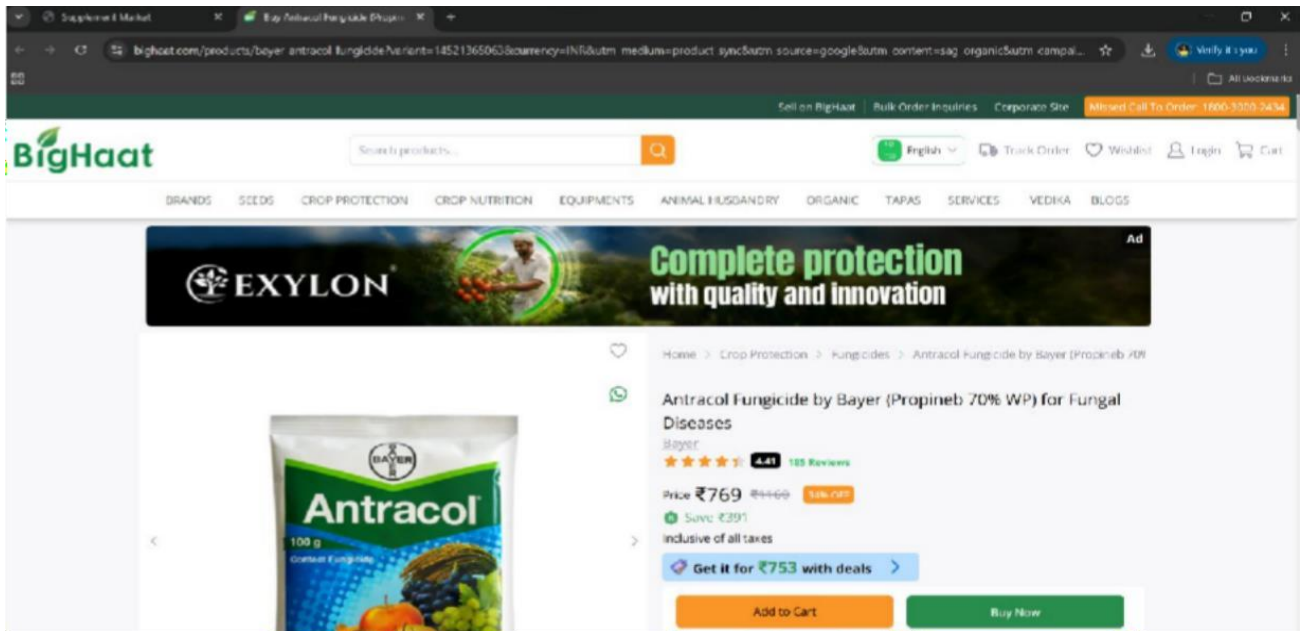
SUBMIT PAGE



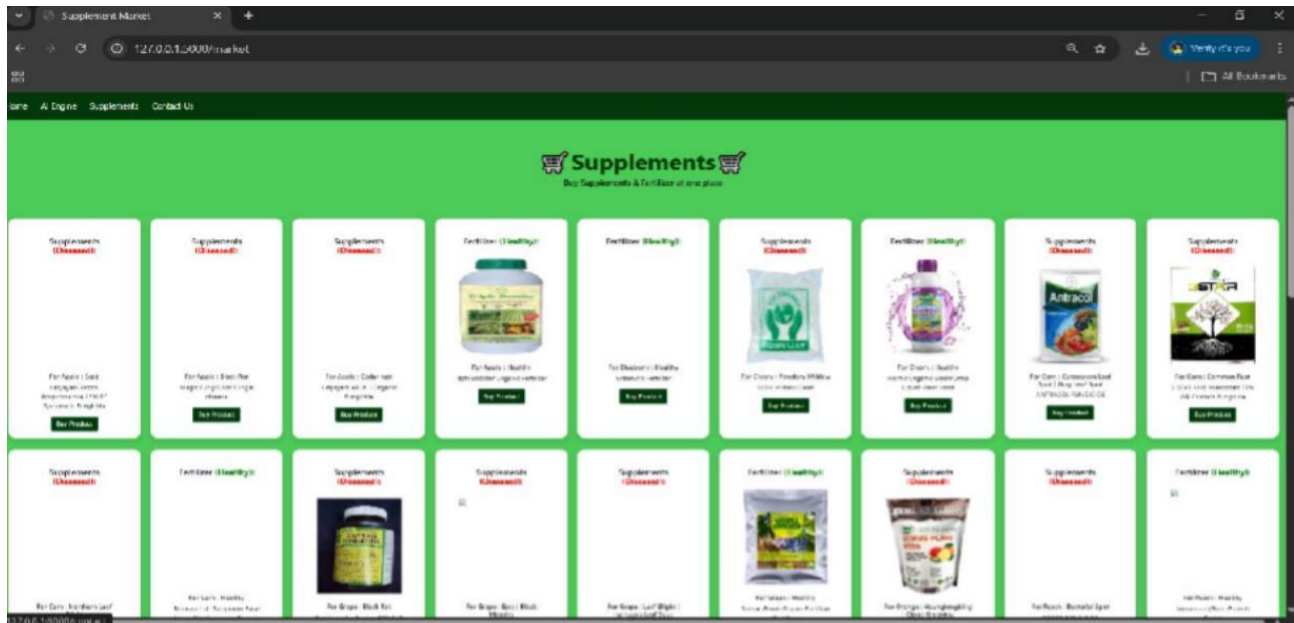
OUTPUT PAGE



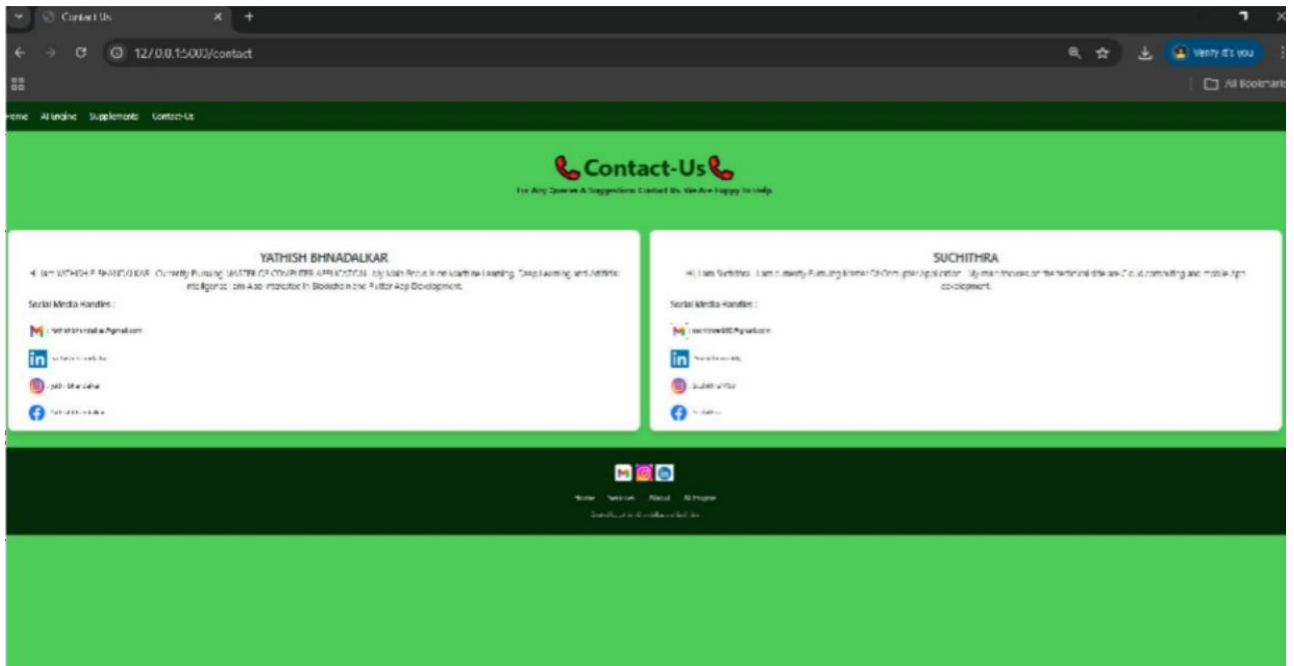
BUYING PAGE



SUPPLEMENTS PAGE



CONTACT - US



7.1 SAMPLE CODE

```

import os

from flask import Flask, redirect, render_template, request from PIL import Image
import torchvision.transforms.functional as TF import CNN
import numpy as np import torch
import pandas as pd

disease_info = pd.read_csv(r'C:\Users\Yathish\Downloads\Plant-Disease-Detection-
main (1)\Plant-Disease-Detection-main\Flask Deployed App\disease_info.csv' ,
encoding='cp1252')
supplement_info = pd.read_csv(r'C:\Users\Yathish\Downloads\Plant-Disease-
Detection-main (1)\Plant-Disease-Detection-main\Flask Deployed
App\supplement_info.csv',encoding='cp1252')

model = CNN.CNN(39)
model.load_state_dict(torch.load(r"C:\Users\Yathish\Downloads\plant_disease_model_
1_lat est.pt"))
model.eval()

def prediction(image_path):
image = Image.open(image_path) image = image.resize((224, 224)) input_data =
TF.to_tensor(image)
input_data = input_data.view((-1, 3, 224, 224)) output = model(input_data)

```

```

output = output.detach().numpy()
index = np.argmax(output)
return index

```

```

app = Flask(__name__)
UPLOAD_FOLDER =
os.path.join('static', 'uploads') if not
os.path.exists(UPLOAD_FOLDER):
os.makedirs(UPLOAD_FOLDER)

```

```

@app.route('/')
def
home_page():
return render_template('home.html')

```

```

@app.route('/contact')
def contact():
return render_template('contact-us.html')

```

```

@app.route('/index')
def ai_engine_page():
return render_template('index.html')

```

```

@app.route('/mobile-device')
def mobile_device_detected_page():
return render_template('mobile-device.html')

```

```

@app.route('/submit', methods=['GET', 'POST'])
def submit():
if request.method == 'POST':

```

```

image = request.files['image']

filename = image.filename

file_path = os.path.join('static/uploads', filename)

image.save(file_path)

print(file_path)

pred = prediction(file_path)

title = disease_info['disease_name'][pred]

description = disease_info['description'][pred]

prevent = disease_info['Possible Steps'][pred]

image_url = disease_info['image_url'][pred]

supplement_name = supplement_info['supplement name'][pred]

supplement_image_url = supplement_info['supplement image'][pred]

supplement_buy_link = supplement_info['buy link'][pred]

return render_template('submit.html' , title = title , desc = description , prevent =
prevent ,

                        image_url = image_url , pred = pred ,sname = supplement_name , simage
= supplement_image_url , buy_link = supplement_buy_link)

@app.route('/market', methods=['GET', 'POST'])
def market():

    return render_template('market.html', supplement_image =
list(supplement_info['supplement image']),

                        supplement_name = list(supplement_info['supplement name']), disease =
list(disease_info['disease_name']), buy = list(supplement_info['buy link']))

if __name__ == '__main__':

    app.run(debug=True)

import pandas as pd

import torch.nn as nn

```

```

class
CNN(nn.Module):
def __init__(self, K):
    super(CNN, self).__init__()
    self.conv_layers = nn.Sequential(
        # conv1
        nn.Conv2d(in_channels=3, out_channels=32,
                    kernel_size=3, padding=1),
        nn.ReLU(),
        nn.BatchNorm2d(32),
        nn.Conv2d(in_channels=32, out_channels=32,
                    kernel_size=3, padding=1),
        nn.ReLU(),
        nn.BatchNorm2d(32),
        nn.MaxPool2d(2),
        # conv2
        nn.Conv2d(in_channels=32, out_channels=64,
                    kernel_size=3, padding=1),
        nn.ReLU(),
        nn.BatchNorm2d(64),
        nn.Conv2d(in_channels=64, out_channels=64,
                    kernel_size=3, padding=1),
        nn.ReLU(),
        nn.BatchNorm2d(64),
        nn.MaxPool2d(2),
        # conv3
        nn.Conv2d(in_channels=64, out_channels=128,
                    kernel_size=3, padding=1),
        nn.ReLU(),
        nn.BatchNorm2d(128),

```

```

nn.Conv2d(in_channels=128,
out_channels=128, kernel_size=3,
padding=1), nn.ReLU(),
nn.BatchNorm2d(128), nn.MaxPool2d(2),
    # conv4
    nn.Conv2d(in_channels=128, out_channels=256,
        kernel_size=3, padding=1),
    nn.ReLU(),
    nn.BatchNorm2d(256),
    nn.Conv2d(in_channels=256, out_channels=256,
        kernel_size=3, padding=1),
    nn.ReLU(),
    nn.BatchNorm2d(256),
    nn.MaxPool2d(2),
)

self.dense_layers =
    nn.Sequential( nn.Dropout(0.4),
    nn.Linear(50176, 1024),
    nn.ReLU(),
    nn.Dropout(0.4),
    nn.Linear(1024, K),
)

def forward(self, X):
    out = self.conv_layers(X)

    # Flatten
    out = out.view(-1, 50176)

```



```
# Fully connected
```

```
out = self.dense_layers(out)
```

```
return out
```

```
idx_to_classes = {0: 'Apple___Apple_scab',
                  1: 'Apple___Black_rot',
                  2: 'Apple___Cedar_apple_rust',
                  3: 'Apple___healthy',
                  4: 'Background_without_leaves',
                  5: 'Blueberry___healthy',
                  6: 'Cherry___Powdery_mildew',
                  7: 'Cherry___healthy',
                  8: 'Corn___Cercospora_leaf_spot Gray_leaf_spot',
                  9: 'Corn___Common_rust',
                  10: 'Corn___Northern_Leaf_Blight',
                  11: 'Corn___healthy',
                  12: 'Grape___Black_rot',
                  13: 'Grape___Esca_(Black_Measles)',
                  14: 'Grape___Leaf_blight_(Isariopsis_Leaf_Spot)',
                  15: 'Grape___healthy',
                  16: 'Orange___Haunglongbing_(Citrus_greening)',
                  17: 'Peach___Bacterial_spot',
                  18: 'Peach___healthy',
                  19: 'Pepper,_bell___Bacterial_spot',
                  20: 'Pepper,_bell___healthy',
                  21: 'Potato___Early_blight',
                  22: 'Potato___Late_blight',
                  23: 'Potato___healthy'}
```

24: 'Raspberry___healthy',
 25: 'Soybean___healthy',
 26: 'Squash___Powdery_mildew',
 27: 'Strawberry___Leaf_scorch',
 28: 'Strawberry___healthy',
 29: 'Tomato___Bacterial_spot',
 30: 'Tomato___Early_blight',
 31: 'Tomato___Late_blight',
 32: 'Tomato___Leaf_Mold',
 33: 'Tomato___Septoria_leaf_spot',
 34: 'Tomato___Spider_mites Two-spotted_spider_mite',
 35: 'Tomato___Target_Spot',
 36: 'Tomato___Tomato_Yellow_Leaf_Curl_Virus',
 37: 'Tomato___Tomato_mosaic_virus',
 38: 'Tomato___healthy'}

7.2 FRONT END

```
<html>
```

```
{% extends 'base.html' %}
```

```
{% block pagetitle %}
```

AI Engine

```
{% endblock pagetitle %}
```

```
{% block body %}
```

```
<div>
```

```
<div class="container">
```

```
<!-- For demo purpose -->
```

```
<div class="row mb-5 text-center text-white">
```

```
<div class="col-lg-10 mx-auto">
```

```
<h1 class="display-4" style="padding-top: 2%;font-weight: 400;color: rgb(4, 54,
```

```
4);"><b> AI
```

Engine </h1>

<p class="lead" style="font-weight: 500;color: black;">Let AI Engine Will Help
You To Detect Disease</p>

</div>

</div>

<!-- End -->

<div class="row ">

<div class="col mx-auto">

<div class="p-5 bg-white shadow rounded-lg" style="height: 95%;">

<h5>Why is it necessary to detect disease in plant ?</h5>

<p>Plant diseases affect the growth of their respective species. In addition, some
research gaps are

identified from which to obtain greater transparency for detecting diseases in
plants, even

before their symptoms appear clearly.

diagnosis is one of the most important aspects of a plant pathologist's training.

Without proper

identification of the disease and the disease-causing agent, disease control
measures can be a

waste of time and money and can lead to further plant losses. Proper disease
diagnosis is

necessary.

</p>

</div>

</div>

<div class="col mx-auto">

<div class="p-5 bg-white shadow rounded-lg" style="height: 95%;">

<!-- Default bootstrap file upload-->

<form action="/submit" method="POST" enctype="multipart/form-data">

<div class="custom-file overflow-hidden mb-4">

<input type="file" id="actual-btn" hidden name="image" />

<label for="actual-btn">Choose File</label>

<label id="camera-btn">Open Camera</label>

</br>

No file chosen

</div>

<!-- Camera feed container (hidden initially) -->

<div id="camera-container" style="display: none;">

<video id="camera-feed" width="320" height="240" autoplay></video>

<button type="button" id="capture-btn">Capture Photo</button>

</div>

<!-- Preview Image (will show the captured photo) -->

<!-- End -->

<h6 class="text-center mb-4 text-muted">

Simply upload your plant's leaf image and then see the magic of AI.

</h6>

```

        <!-- Custom bootstrap upload file-->

        <center>

            <a class="mx-2"><button type="submit" class="btn btn-outline-
success">Submit</button></a>

        </center>

    </form>

    <!-- End -->

</div>

</div>

<div class="col mx-auto">

    <div class="p-5 bg-white shadow rounded-lg" style="height: 95%;">

        <h5><b>Prevent Plant Disease follow below steps:</b></h5>

        <ol>

            <li>Follow Good Sanitation Practices.</li>

            <li>Fertilize to Keep Your Plants Healthy.</li>

            <li>Inspect Plants for Diseases Before You Bring Them Home.</li>

            <li>Allow the Soil to Warm Before Planting.</li>

            <li>Ensure a Healthy Vegetable Garden By Rotating Crops.</li>

            <li>Provide Good Air Circulation</li>

            <li>Remove Diseased Stems and Foliage</li>

        </ol>

        <a target="_blank" href="https://www.thespruce.com/prevent-plant-diseases-in-
your-garden-2539511"

            class="mx-2"><button type="button" class="btn btn-outline-success">More
info</button></a>

    </div>

</div>

</div>

</div>

```

```
</div>
```

```
<script>
```

```
const actualBtn = document.getElementById('actual-btn');
const captureBtn = document.getElementById('capture-btn');

const fileChosen = document.getElementById('file-chosen');

actualBtn.addEventListener('change', function ()
  { fileChosen.textContent = this.files[0].name
  })
```

7.3 BACKEND

```
let capturedFile = null; // Variable to hold the captured file

document.getElementById('camera-btn').addEventListener('click', function () {
  // Show the camera container
  document.getElementById('camera-container').style.display = 'block';

  // Start the camera feed
  startCamera();
});

// Start the camera feed using the getUserMedia API
function startCamera() {
  const cameraFeed = document.getElementById('camera-feed');

  // Request access to the user's camera
  navigator.mediaDevices.getUserMedia({ video: true })
    .then(function (stream) {
```

```

    // Set the camera feed source to the stream
    cameraFeed.srcObject = stream;
  })
  .catch(function (error) {
    console.log('Error accessing the camera: ', error);
  });
}

// Capture photo from the camera feed
document.getElementById('capture-btn').addEventListener('click', function ()
{
  const cameraFeed = document.getElementById('camera-feed');
  const canvas = document.createElement('canvas');
  const ctx = canvas.getContext('2d');

  // Set canvas dimensions to match the video feed dimensions
  canvas.width = cameraFeed.videoWidth;
  canvas.height = cameraFeed.videoHeight;

  // Draw the current frame from the video feed onto the canvas
  ctx.drawImage(cameraFeed, 0, 0, canvas.width, canvas.height);

  // Convert the canvas image to a data URL
  const dataUrl = canvas.toDataURL('image/jpeg');

  // Create a new file from the data URL
  const imageBlob = dataURIToBlob(dataUrl);
  capturedFile = new File([imageBlob], "camera_image.jpg", { type: 'image/jpeg' });

  // Now you have a file object that you can use locally
  document.getElementById('file-chosen').textContent = capturedFile.name;

```

```
// Show the captured image preview
const preview = document.getElementById('preview');
preview.src = dataUrl;
preview.style.display = 'block'; // Show the preview
});
```

```
// Helper function to convert data URL to Blob
function dataURItoBlob(dataURI) {
  const byteString = atob(dataURI.split(',')[1]);
  const arrayBuffer = new ArrayBuffer(byteString.length);
  const uintArray = new Uint8Array(arrayBuffer);
  for (let i = 0; i < byteString.length; i++)
    { uintArray[i] = byteString.charCodeAt(i);
  }
  return new Blob([uintArray], { type: 'image/jpeg' });
}
```

```
// Set up the "Choose File" input to trigger when the image is captured
document.getElementById('actual-btn').addEventListener('click', function () {
  // Use the captured image file as the input file for form submission
  if (capturedFile) {
    const dataTransfer = new DataTransfer(); // Create a DataTransfer object
    dataTransfer.items.add(capturedFile); // Add the captured file to the DataTransfer
object
    document.getElementById('actual-btn').files = dataTransfer.files; // Assign the file to
the input
```


CHAPTER 8

CONCLUSION

In this paper, we present Image recognition using a convolution neural network where the user's uploaded leaves are matched with trained data using CNN, where pre-processing is much lesser compared to other processes like Distance metric learning and matrix factorization. Even though CNN has high accuracy in image recognition problems, CNN does not encode the position and orientation of the object. Lack of ability to be spatially invariant to the input data and lots of training data is required. We leave these constraints for future enhancement where the application should encode the position and or orientation of the object and work efficiently with minimal trained data.

Plant leaf disease detection using Convolutional Neural Networks (CNNs) has emerged as an effective and reliable approach for identifying plant diseases at an early stage. By automatically extracting features from leaf images, CNN models eliminate the need for manual feature engineering and provide high accuracy in classification tasks. This technology helps farmers and agricultural experts quickly diagnose diseases, reduce crop loss, and improve overall productivity. The integration of image processing with deep learning makes the system scalable, efficient, and suitable for real-time applications.

In conclusion, CNN-based plant disease detection systems play a crucial role in advancing smart agriculture by enabling timely intervention and better crop management practices. With further improvements such as larger datasets, better model optimization, and deployment through mobile or web applications, these systems can become more accessible and widely adopted. Ultimately, this approach contributes to sustainable agriculture, ensuring food security and supporting farmers in making informed decisions.

CHAPTER 9

FUTURE ENHANCEMENT

Future enhancements in plant leaf disease detection using CNNs can focus on improving model accuracy and generalization by training on larger and more diverse datasets. Including images captured under different lighting conditions, backgrounds, and growth stages will help the model perform better in real-world scenarios. Advanced architectures and techniques such as transfer learning, data augmentation, and hybrid deep learning models can further enhance detection performance and reduce overfitting.

Another important direction is the integration of the system into mobile and edge-based applications. Developing lightweight CNN models that can run efficiently on smartphones or low-power devices will make the technology more accessible to farmers in remote areas. Real-time disease detection through mobile apps, combined with features like offline functionality and multilingual support, can significantly improve usability and adoption. Additionally, integrating cloud computing can enable faster processing and centralized data storage for continuous model improvement.

Finally, future systems can be expanded to provide comprehensive agricultural support beyond disease detection. By combining CNN-based detection with recommendation systems, farmers can receive suggestions for suitable fertilizers, pesticides, and treatment methods. Integration with IoT sensors for monitoring soil moisture, temperature, and humidity can further enhance decision-making. Such a holistic smart farming approach will not only detect diseases but also prevent them, ultimately leading to increased crop yield, reduced costs, and sustainable agricultural practices.

CHAPTER 10

BIBLIOGRAPHY

1. Ahmed, K, Shahidi, T.R., Irfanul Alam, S.M., Momen, S., 2019. Rice leaf disease detection using Machine Learning techniques, 2019 International Conference on Sustainable Technologies for Industry 4.0 (STI). IEEE, Dhaka, Bangladesh.
2. Taskeen Ashraf, Yasir Niaz Khan., 2020. Weed density classification in rice crop using computer vision. Compute, Electron, Agric.175.
3. Sanga, S.L., Machuve, D., Jomanga, K., 2020. Mobile-based deep learning models for Banana Disease detection. Technol. Appl. Sci. Res. 10(3).
4. Tewari, V.K., Pareek, C.M., Lal, G., Dhruv, L.K., Singh, N., 2020. Image processing based real-time variable-rate chemical spraying system for diseases control in paddy.
5. Artificial. Intellig. Agric. Ramesh, S., Vydeki, D., 2020. Recognition and classification of paddy leaf diseases using optimized deep neural network with Jaya algorithm.